

Presented by [Gautham Vasan](#)

Robots That Learn on the Fly From Real-World Interaction

Sept 18th, 2026



Animals learn from experience

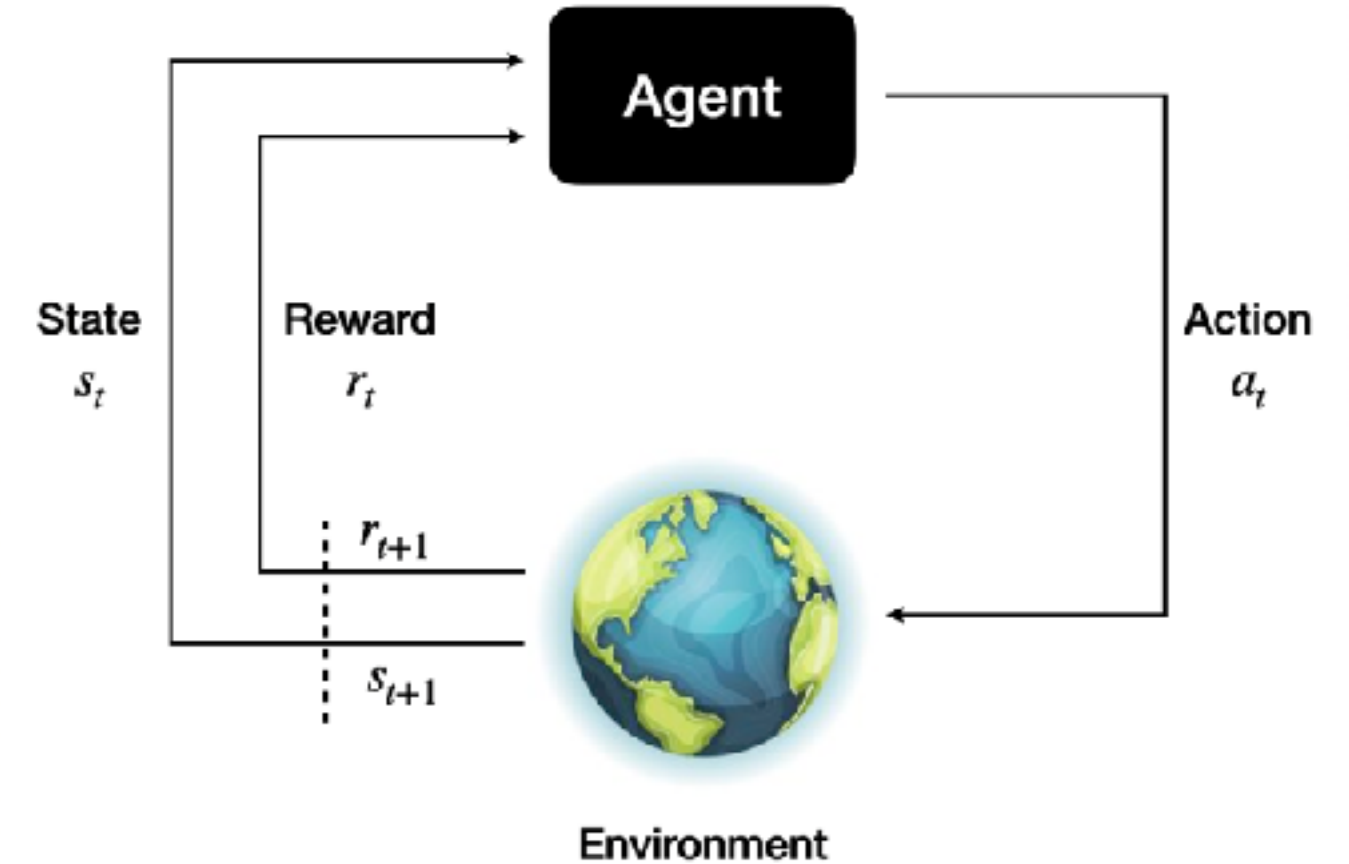
- Animals react in milliseconds to achieve their goals.
- They learn from experience, in real time, as they interact with the world.
- They learn with bounded computation: a nervous system and nothing more.



Rat learns to drive

Real-Time Reinforcement Learning

- In reinforcement learning (RL), an agent learns which actions yield the most reward through trial-and-error interaction with its environment.
- *Real-time RL* is the setting in which an agent senses, acts, and learns from runtime experience as wall-clock time advances for both the agent and the environment.
- The world does not *pause* while the agent computes an action or makes a learning update.



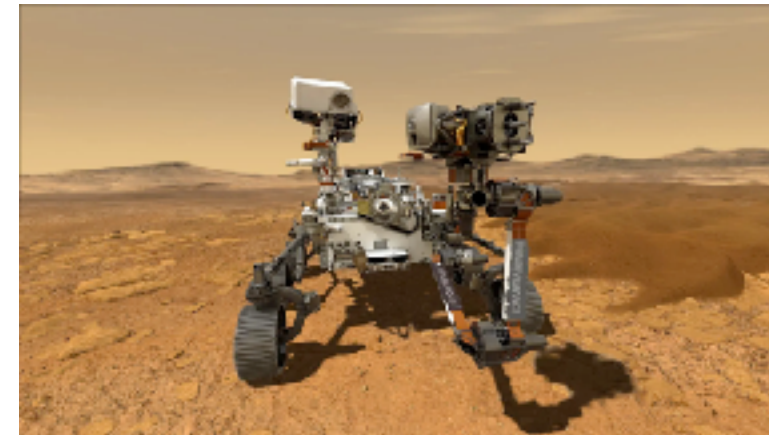
Deployed robots do not learn from runtime experience



Boston Dynamics Spot



iRobot Roomba

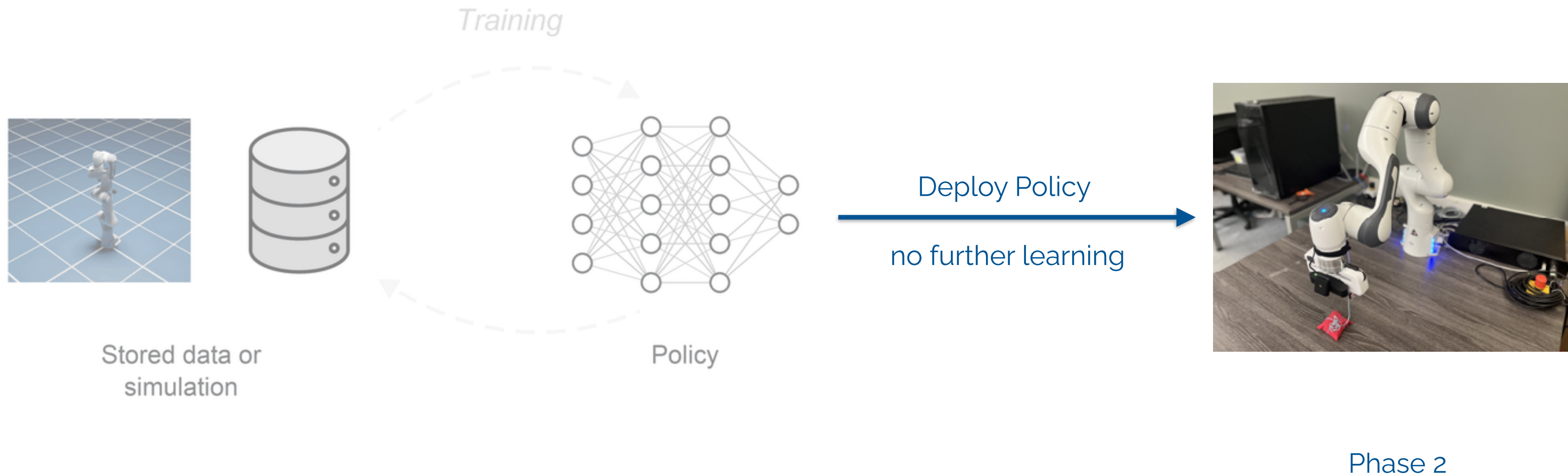


Mars Perseverance Rover



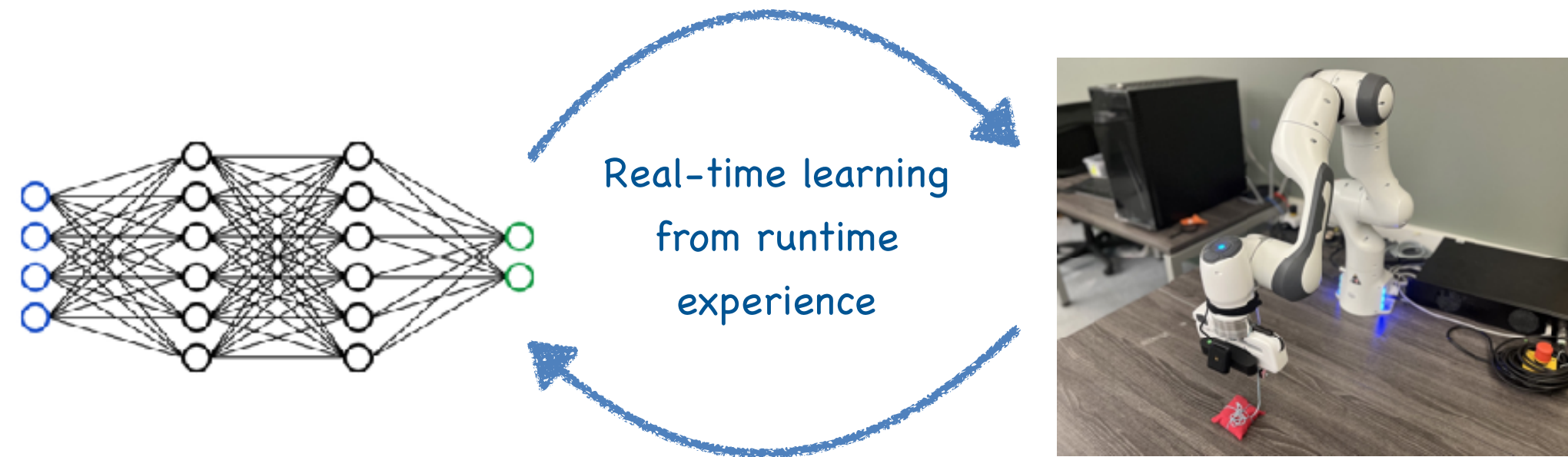
Waymo Self-Driving Car

Two-phase workflows: train, then deploy



- Training and deployment are separated by design.
- Their behavior changes only when engineers collect data, retrain, and redeploy.

Long-term autonomy requires learning during deployment



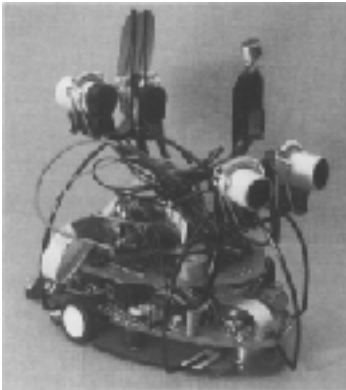
- For robots operating over long periods in changing environments, it is not possible to anticipate and train ahead of deployment for every conceivable scenario.
- Real-time RL provides a mechanism for updating behavior after deployment, reducing reliance on humans for data collection, retraining, and redeployment.

Real-time RL on physical robots has a long history in academic research

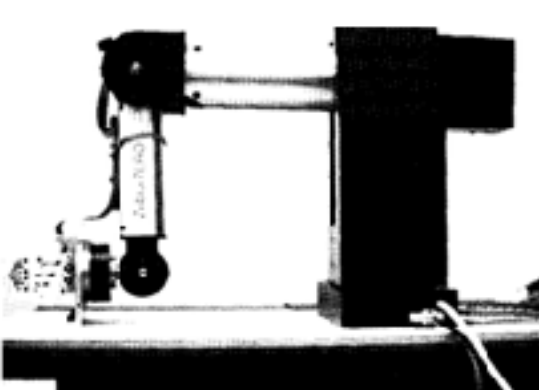
1990s



Mahadevan & Connell (1991)



Dorigo & Colombetti (1994)



Gullapalli et al. (1994)

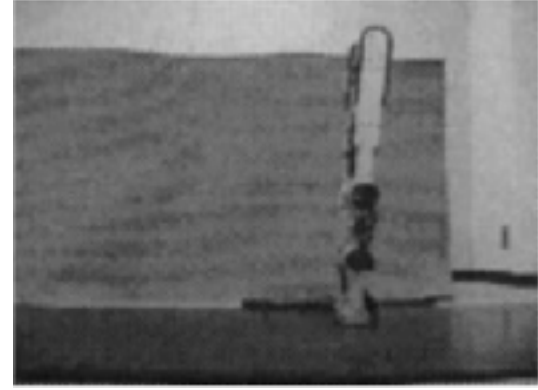


Millan (1996)



Gaskett et al. (2000)

2000s



Morimoto & Doya (2001)

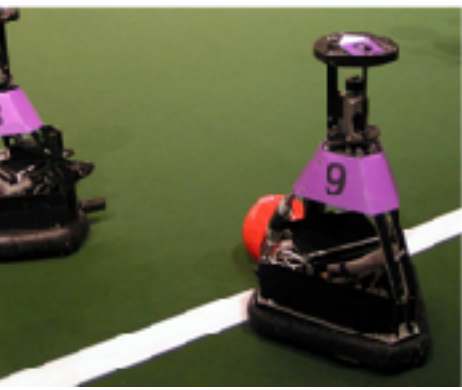


Kohl & Stone (2004)

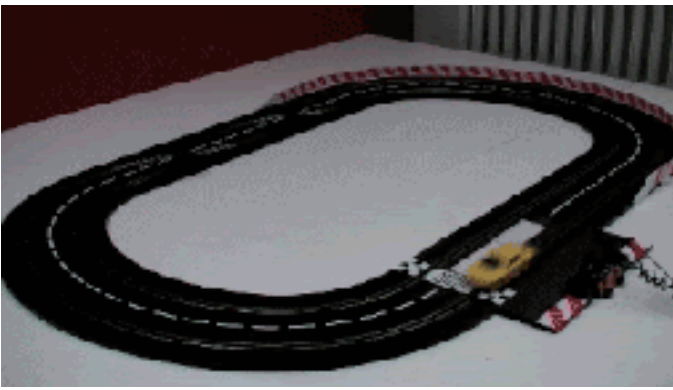


Riedmiller et al. (2007)

2010s



Riedmiller et al. (2009)



Lange et al. (2010)



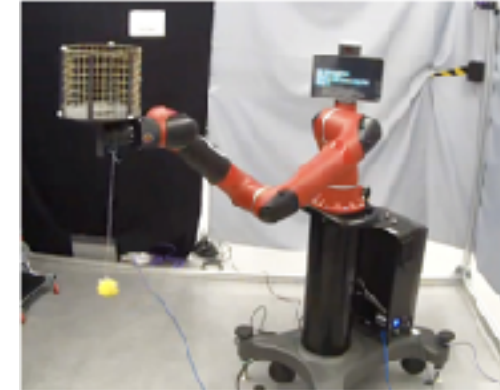
Hester et al. (2012)



Vasan & Pilarski (2017)



Mahmood et al. (2018)



Hertweck et al. (2020)

2020s



Smith et al. (2023)

Real-time RL on physical robots has a long history in academic research

1990s

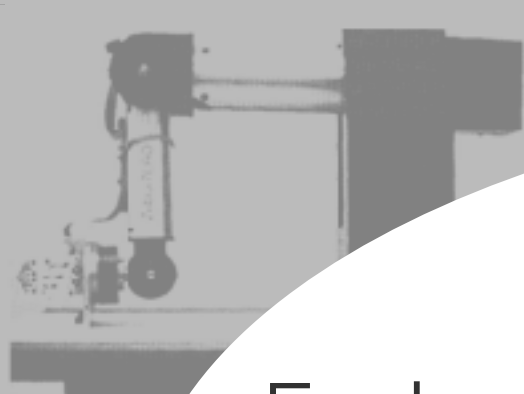
2000s



Mahadevan & Connell
(1991)



Dorigo & Colombetti
(1994)



Gullerud et al. (1995)



Arkin & Stone (2004)



Riedmiller et al. (2007)

2010s

2020s



Riedmiller et al. (2009)



Lange et al. (2010)



Hester et al. (2012)



Vasan & Pilarski (2017)



Mahmood et al. (2018)



Hertweck et al. (2020)



Smith et al. (2023)

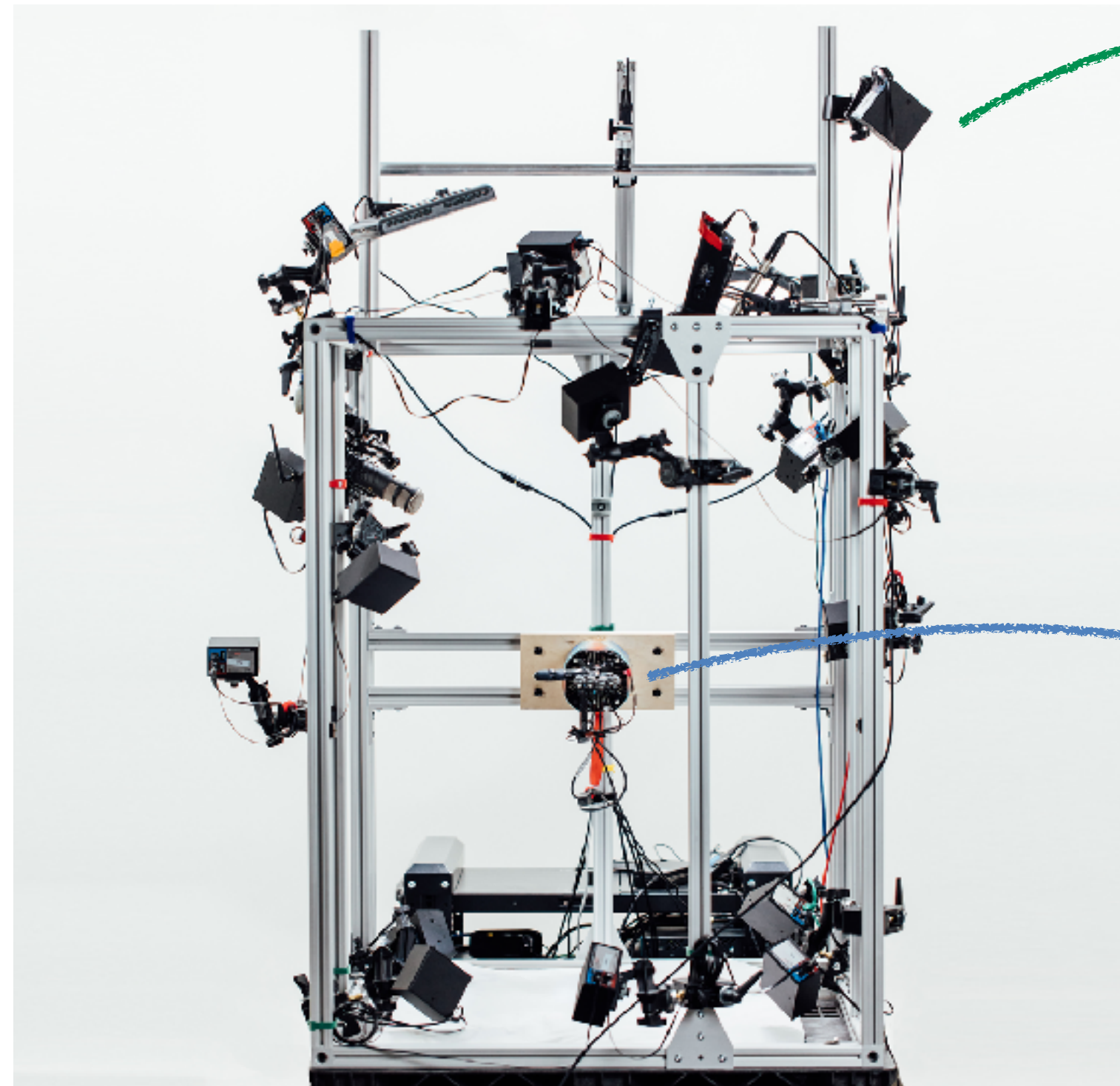
Each of these robotic systems learned in real time, from real-world interaction. Yet real-time RL never became the norm.

Real-time RL on physical robots is hard: it imposes three practical demands that train-then-deploy workflows do not

01

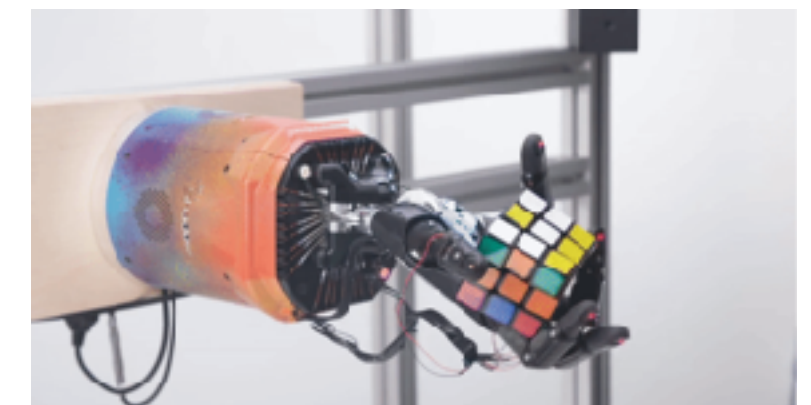
learning must use rewards computable from the robot's own sensing and simple to specify across tasks

Specifying a reward that captures the desired behavior is hard



External instrumentation here includes the depth and motion capture cameras.

The agent only controls this dexterous hand



OpenAI's Solving a Rubik's Cube With a Dexterous Hand
(Akkaya et al. 2019)

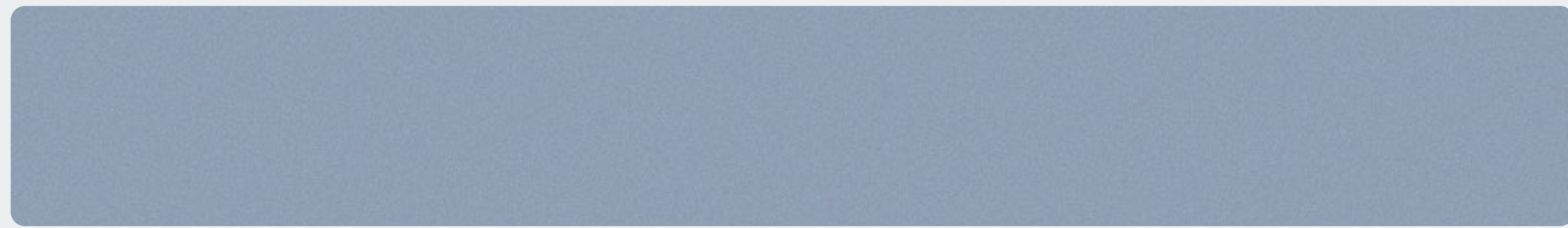
Specifying rewards for a long-lived robot

- A reward signal that depends on external instrumentation or human feedback does not scale during robot operation.
 - Rewards computed from onboard sensing alone free the robot from that dependence.
- A long-lived robot would also be expected to solve many tasks over its lifetime, and engineering a reward function for every task by hand scales poorly.
 - Rewards that are simple to specify limit the per-task effort.



Boston Dynamics Spot
during deployment for site inspection

Real-time RL on physical robots is hard: it imposes three practical demands that train-then-deploy workflows do not



learning must reach competent performance reliably within a practical wall-clock budget

Wall-Clock Budget for Learning

- Learning on physical robots is costly
 - Experience consumes equipment time and energy, hardware wears down with use, and poor exploration risks damage.
- Learning should reach competent performance within a reasonable amount of time.
 - This should hold across repeated runs with fixed hyperparameters
- Both run-to-run consistency and final performance matter.

Real-time RL on physical robots is hard: it imposes three practical demands that train-then-deploy workflows do not

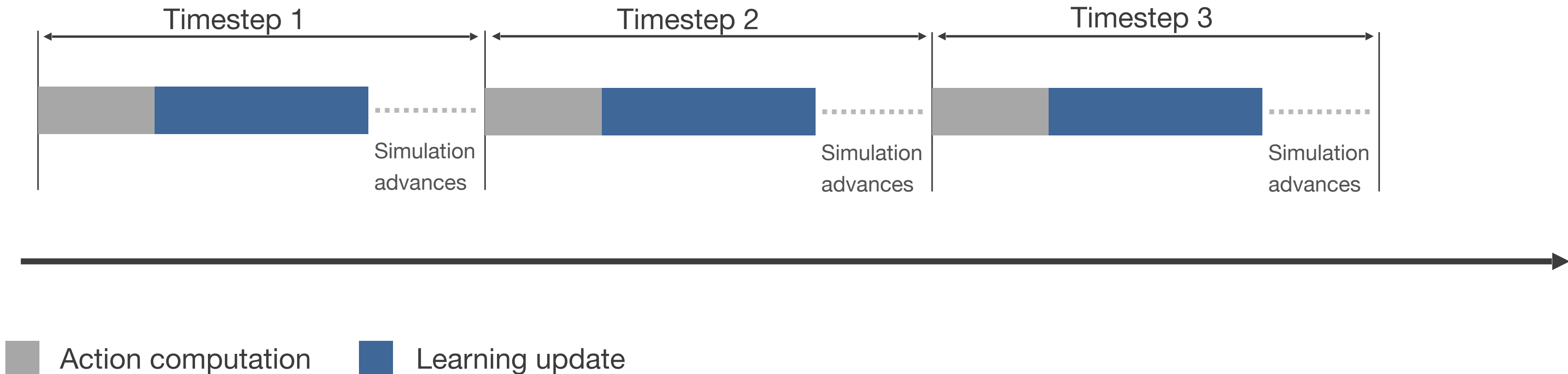


learning must operate within the robot's limited computational and memory resources

Learning must share the robot's limited computation and memory

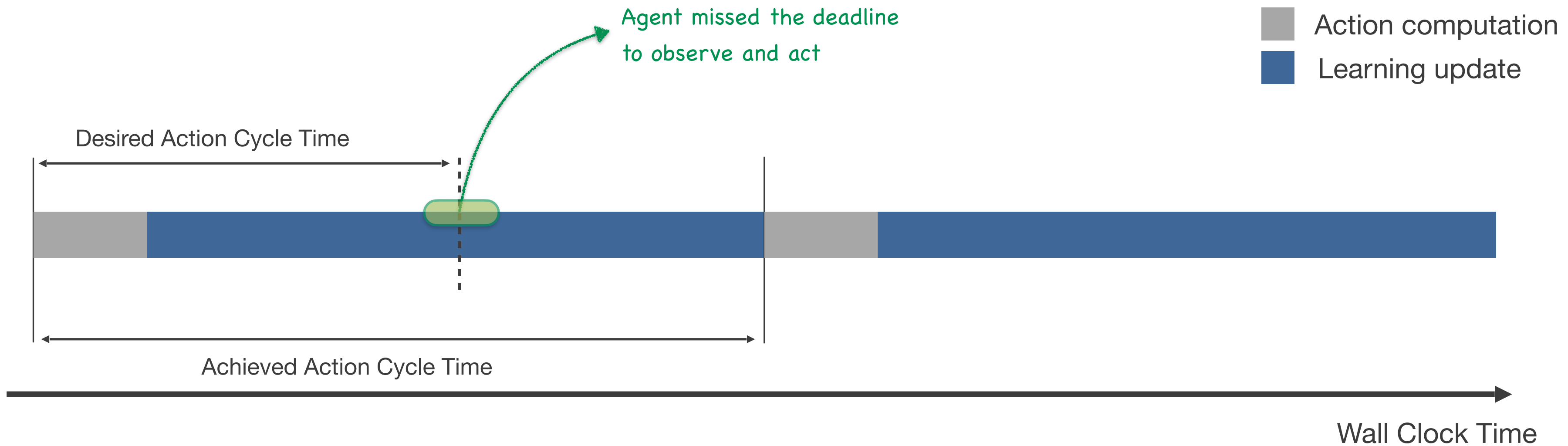
- Learning shares the robot's limited computation and memory with sensing, acting, and other processes.
- Available memory and computation determines the replay buffer capacity, the minibatch size, and the time each learning update requires.
- Each action must be computed within the *action cycle time*, the fixed period between consecutive action selections.
- Slower or infrequent learning updates delay adaptation.

Simulation: the world waits while the agent computes



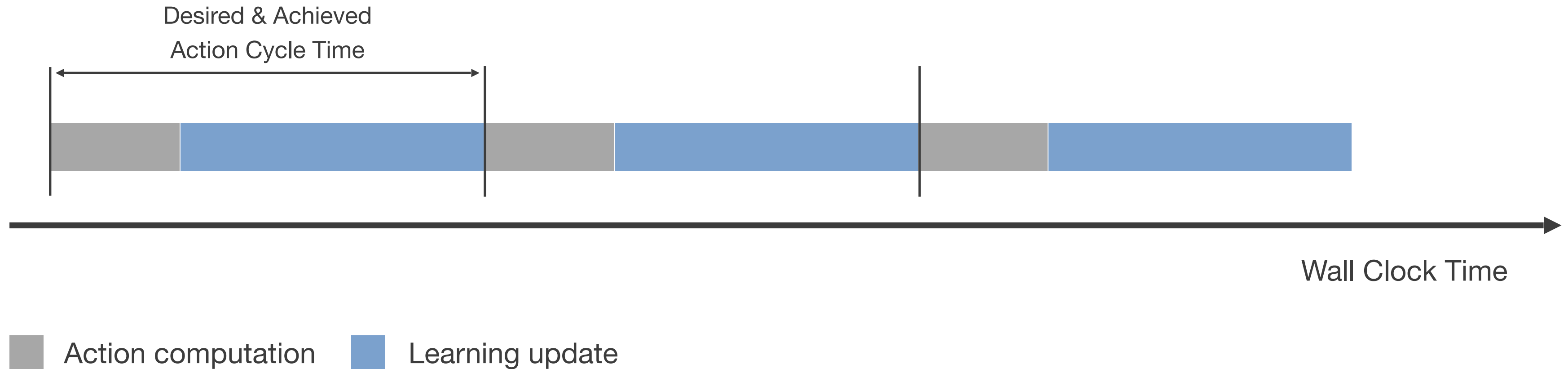
- The agent takes as long as it needs; the world pauses until the update finishes.
- One update per timestep is therefore always affordable, however expensive the update.

On a physical robot, an expensive update stretches the action cycle



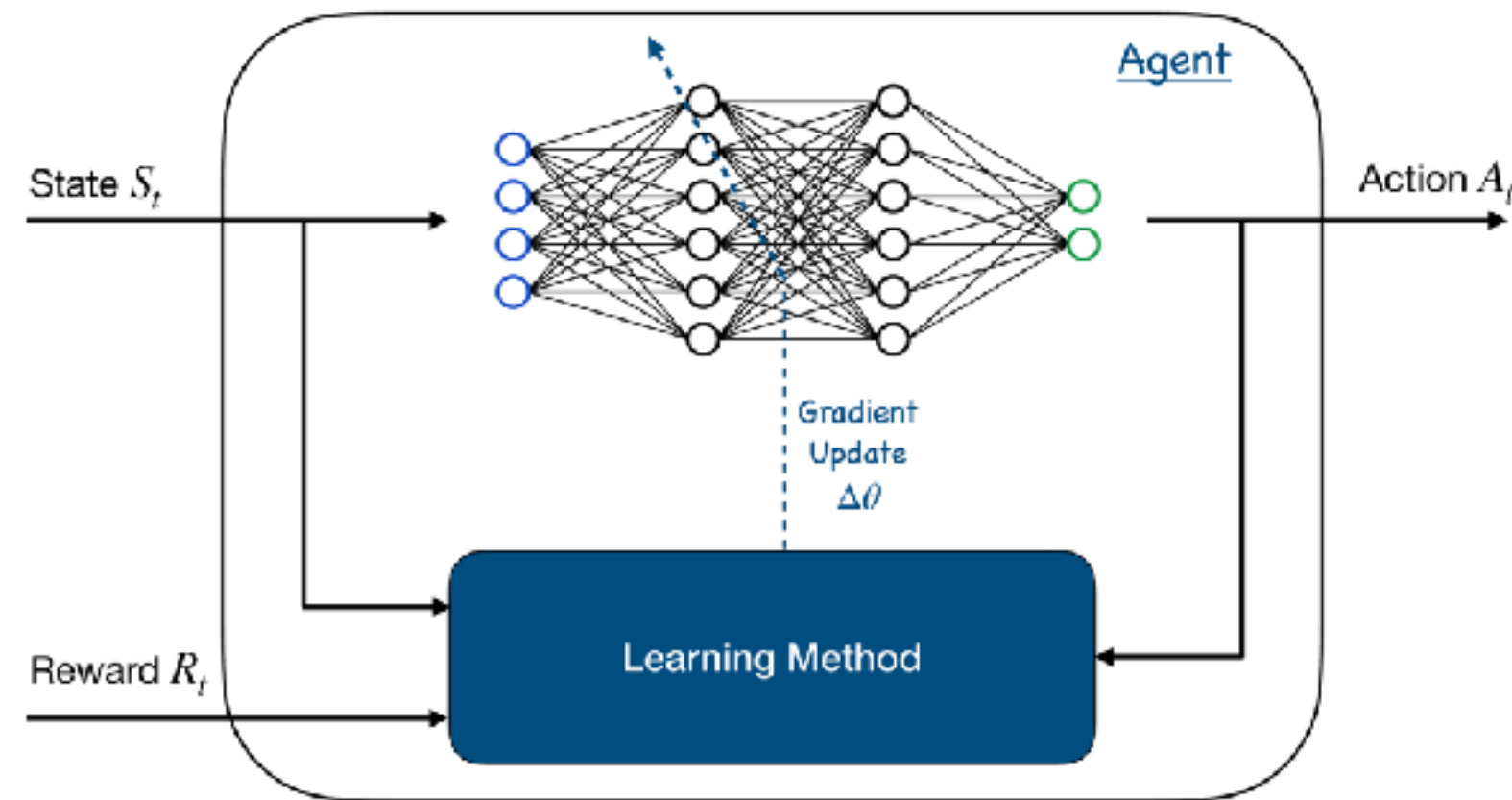
- The world advances whether or not the update has finished.
- An update longer than the desired action cycle results in slow reaction times.

Option 1: Cheap Learning Updates



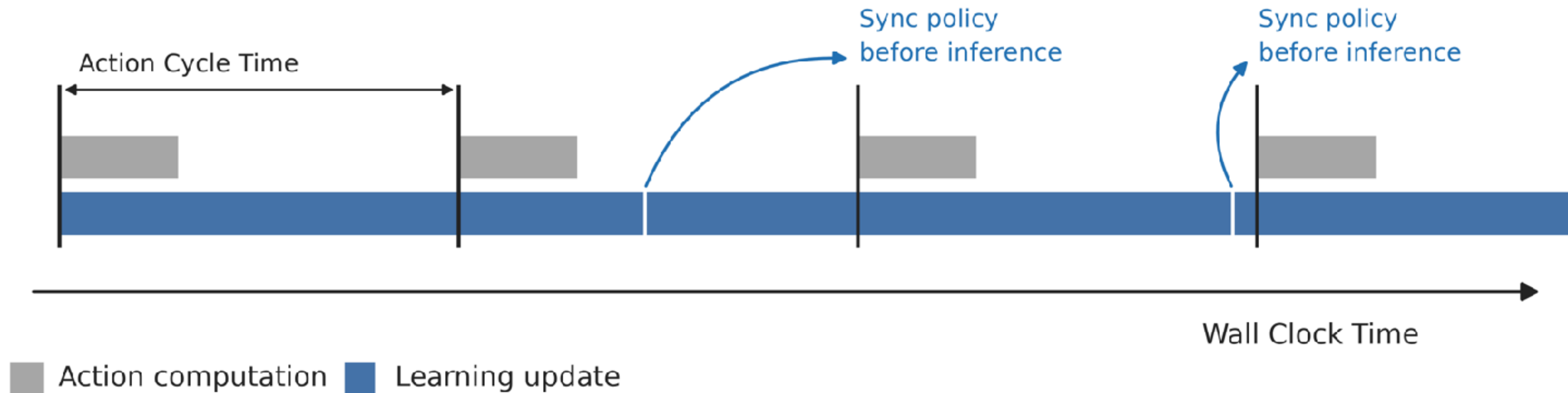
- A cheap update fits inside the action cycle, so the desired and achieved cycle times coincide.
- *Streaming RL* methods take this option: one update per timestep, using only the latest transition.

Streaming Reinforcement Learning



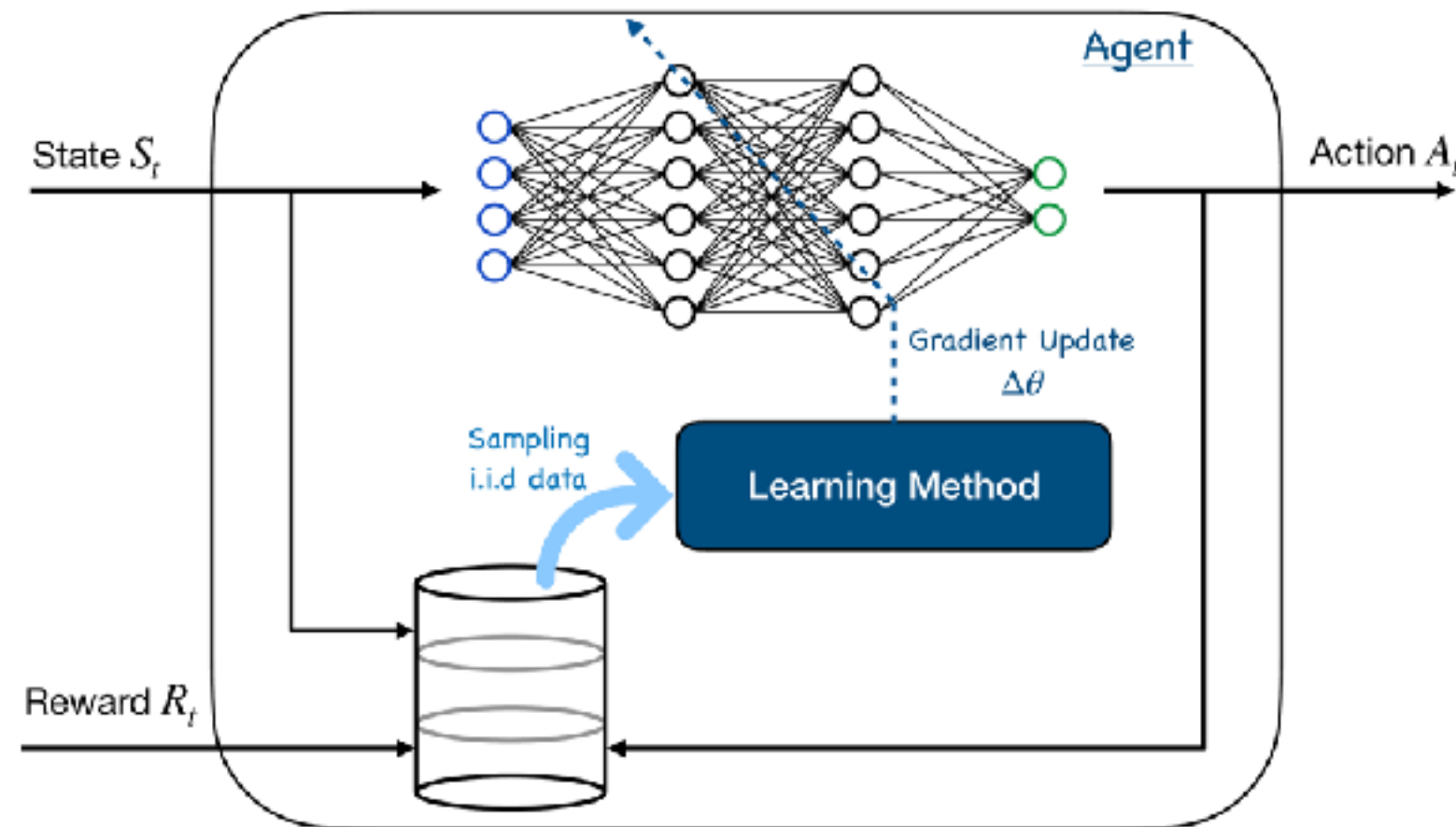
- A streaming RL method learns from a stream of experience, one transition at a time, and stores no transitions in raw form.
- A transition is an observation, an action, a reward, and the next observation.
- Early demonstrations of real-time learning on physical robots utilized streaming RL methods with linear function approximation
- *Streaming deep RL methods*, which use artificial neural networks, are brittle: agents often fail to learn or diverge.

Option 2: Asynchronous, Expensive Learning Updates



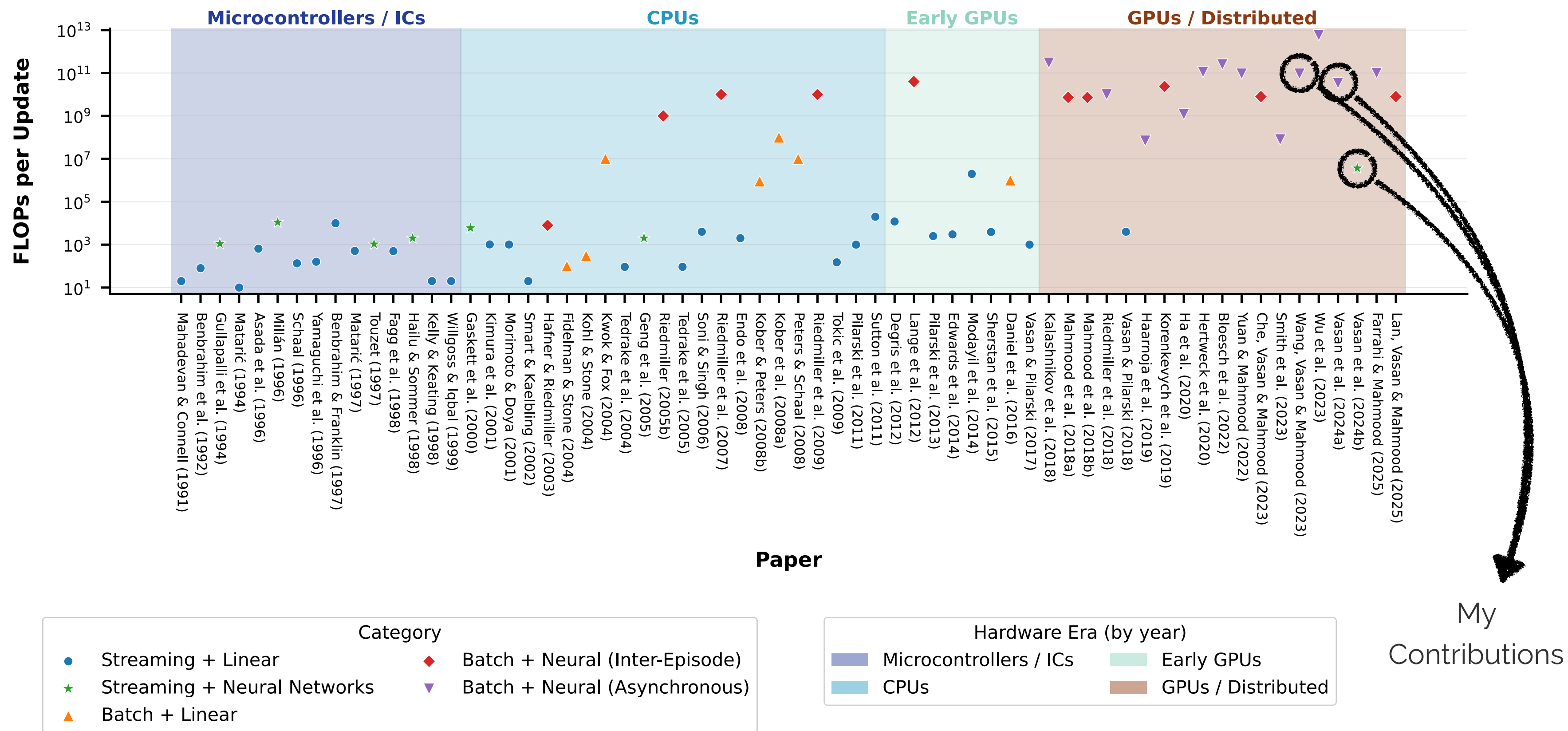
- The learning update runs in its own process, alongside acting, and never blocks the action cycle.
- Before each action, the acting process copies the latest policy parameters.

Batch Reinforcement Learning



- In batch reinforcement learning, the agent stores transitions and computes parameter updates from minibatches of stored experience.
- The success of modern deep RL applications rely on batch updates and large replay buffers.
- Averaging gradient updates over minibatches also lowers their variance and makes learning more stable.
- The stability of batch updates comes at a cost in memory and computation.

Prior work tried both options; recent work favors asynchronous batch updates



Prior work tried both options; recent work favors asynchronous batch updates.

- Early systems used streaming updates with linear function approximation, which fit the action cycle but need hand-designed features that do not scale to high-dimensional inputs.
- Neural networks handle such inputs, but agents using streaming deep RL methods often failed to learn or diverged.
- As computation grew cheaper with Moore's law, the community moved to batch updates with neural networks, which are stable but too expensive for a robot's onboard computer.

My Contributions



ReLoD:

A computational framework for real-time RL on robots using two computers



An empirical study of minimum-time task specification (-1 reward until termination) for goal-reaching tasks



AVG:

A streaming deep RL method for continuous control

Contribution #1: ReLoD

A Remote-Local Distributed framework for
vision-based real-time RL on real robots

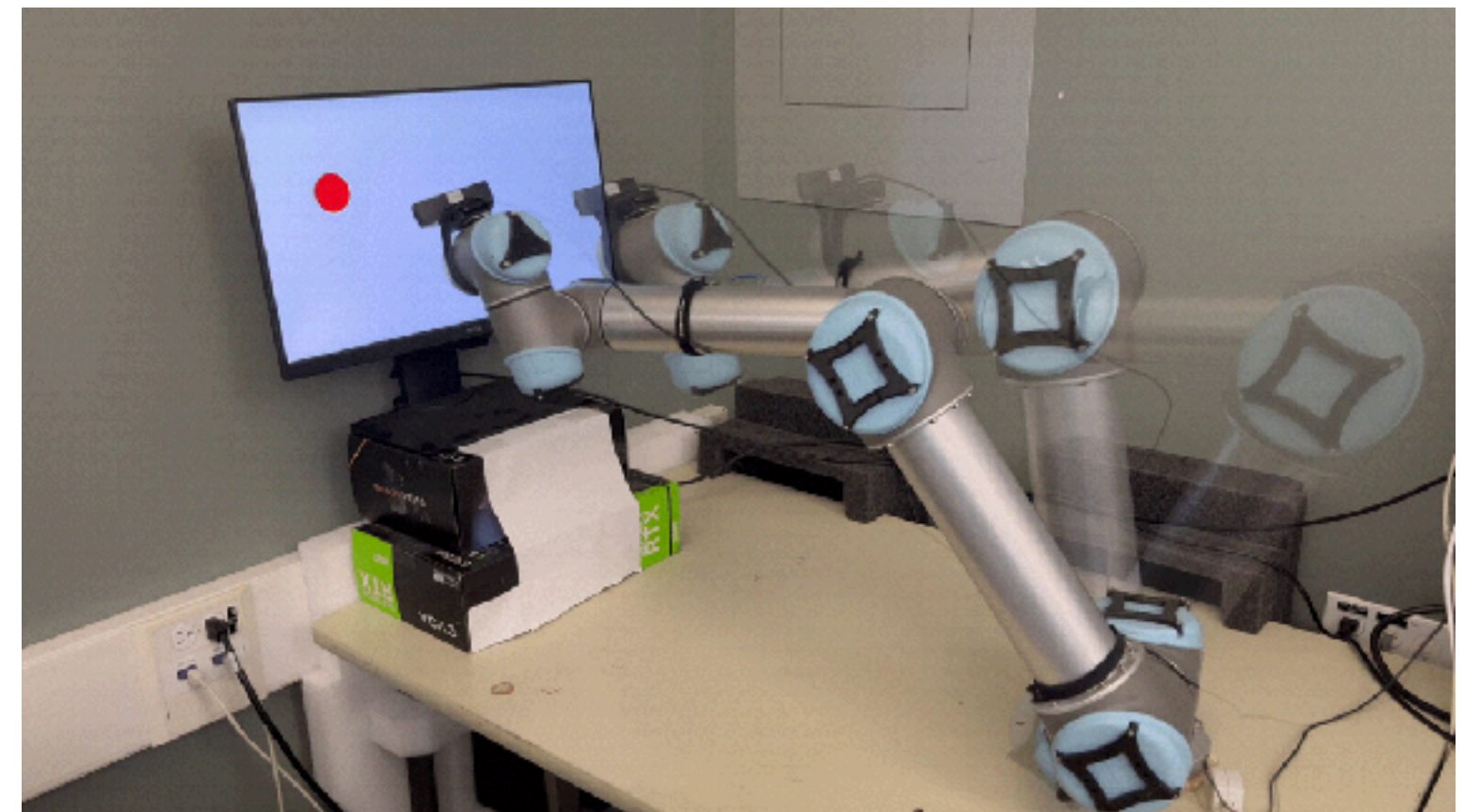
Wang*, Vasan* & Mahmood (2023); ICRA

Why ReLoD?

- Modern deep RL methods that learn from pixels rely on batch updates too expensive for a robot's onboard computer.
- ReLoD supplements the onboard computer with a remote workstation, dividing the agent's computation between the two.
- With ReLoD, a robot acts on its own computer and learns on the workstation, at no loss in performance in our experiments.

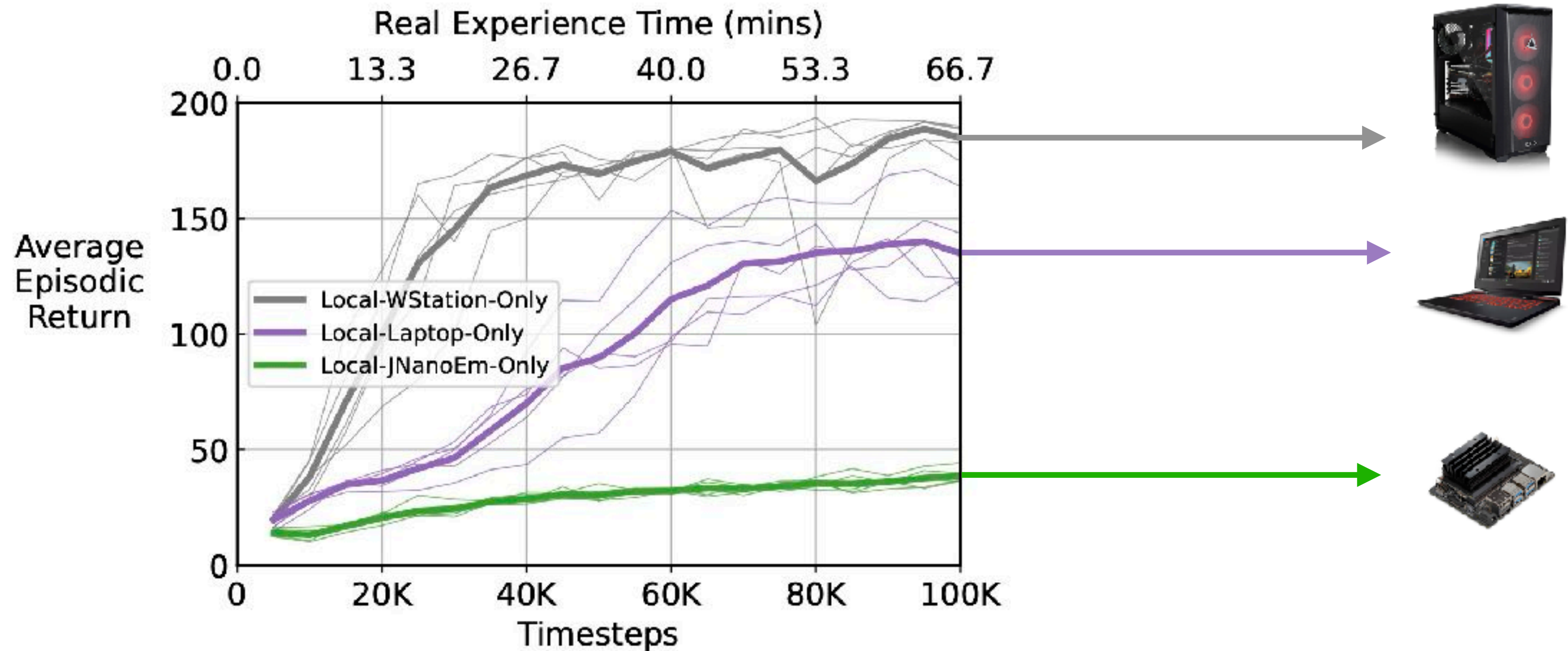
UR5-VisualReacher Task

- A UR5 arm moves its fingertip to a random target, a red circle on a monitor.
- Observation: three consecutive 160×90 camera images, joint angles, joint velocities, and the previous action.
- Action: desired angular velocities of five joints, sent every 40 ms.
- Reward: moving the fingertip toward the target and keeping it centered in the image; abrupt joint twists are penalized.
- Episode length: 100 timesteps, or 4 seconds.



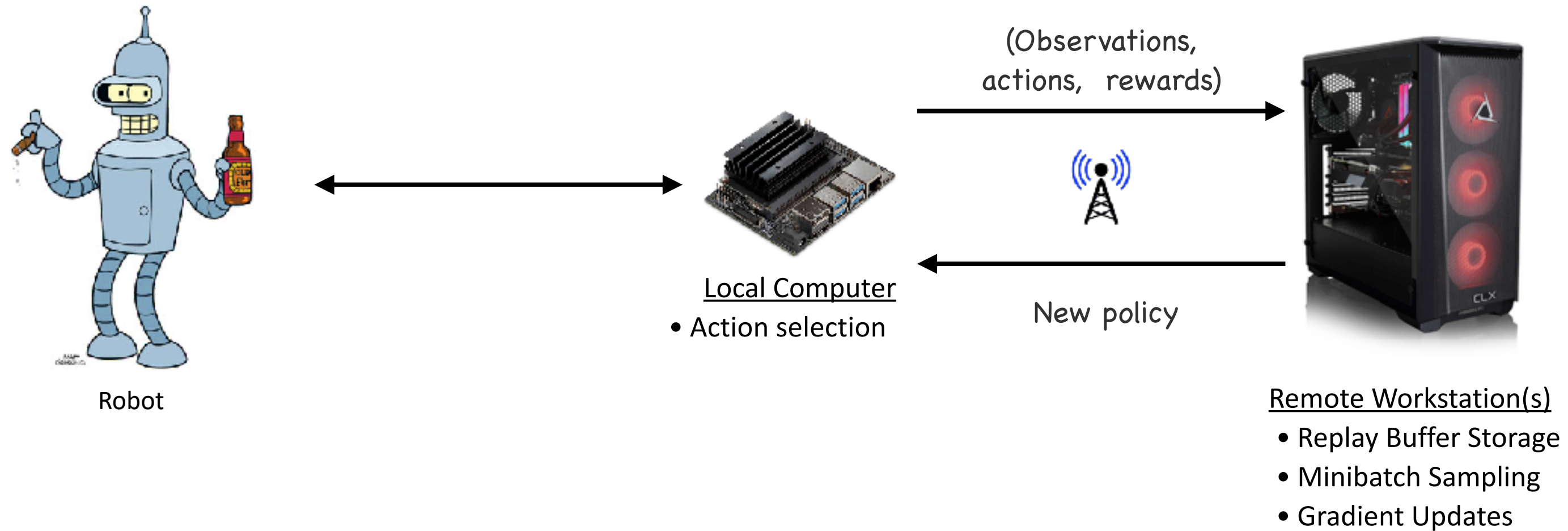
UR5-VisualReacher Task

Learning with Soft Actor-Critic (SAC) degraded as onboard computation shrank



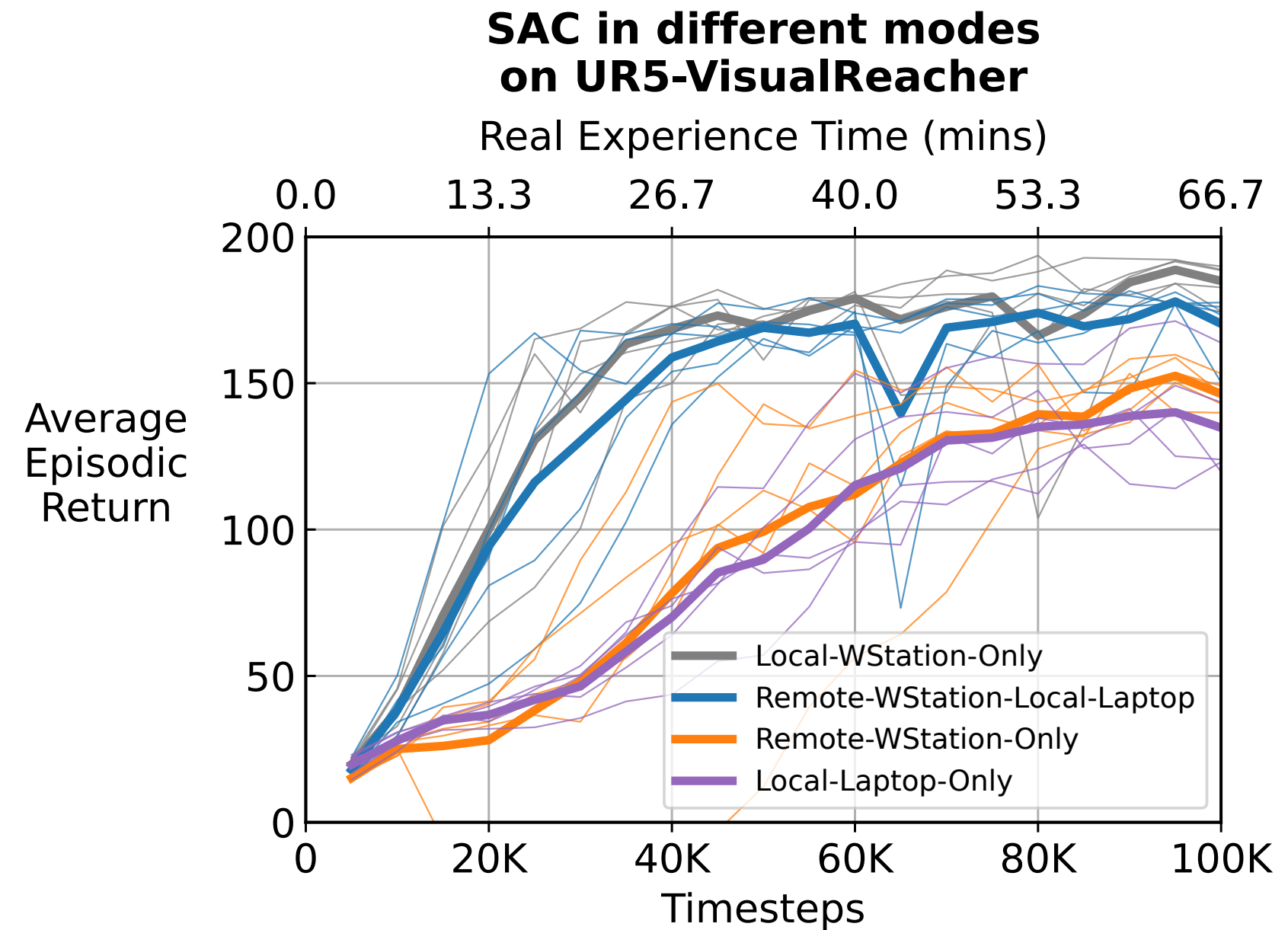
- As onboard computation shrinks, replay buffer capacity and mini batch size change accordingly.
- Thick lines are means over 5 runs; thin lines are individual runs.

ReLoD: The Remote Local Distributed Framework



- ReLoD distributes an agent's learning computations between the robot's local computer and a remote workstation.
- Three modes place the computation: Remote-Only, Remote-Local, and Local-Only.

Remote
updates
recovered
performance;
remote action
computation
added delay



- In **Remote-Local** mode, the robot's computer samples actions and the workstation computes updates.
- In **Remote-Only** mode, the workstation does both, and every action travels over Wi-Fi before execution.

Create-Reacher, learned with ReLoD



Random Policy (1x)



iRobot Create2



Jetson Nano



WiFi



Remote Workstation

Summary

- Learning with SAC degraded as onboard computation shrank, from workstation to Jetson Nano.
- ReLoD lets the robot act locally and learn remotely.
- Remote-Local learning matched learning on a workstation alone on UR5-VisualReacher.
- Learning still depends on high-bandwidth communication with a remote computer.

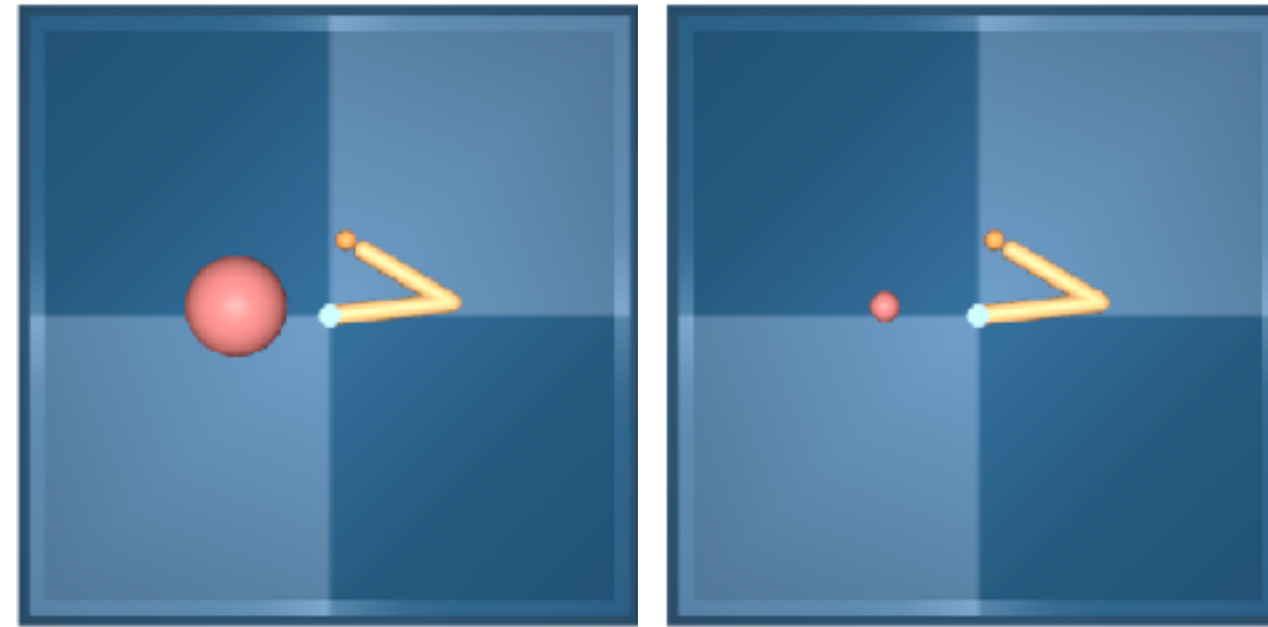
Contribution #2:
An empirical study of the
minimum-time specification
for goal-reaching tasks

Vasan et al. (2024a); RLC

Why the minimum-time specification?

- The minimum-time specification: a reward of -1 at every timestep until the robot reaches the goal.
- Only recognition of success is required, so one specification serves every goal-reaching task.
- The informative signal for learning is sparse, which the community perceives as hard to learn from.
- On the goal-reaching tasks studied, agents learned, and their final policies matched or outperformed those under engineered rewards that guide the robot toward the goal.

The Reacher Task



Reacher-Easy

Reacher-Hard

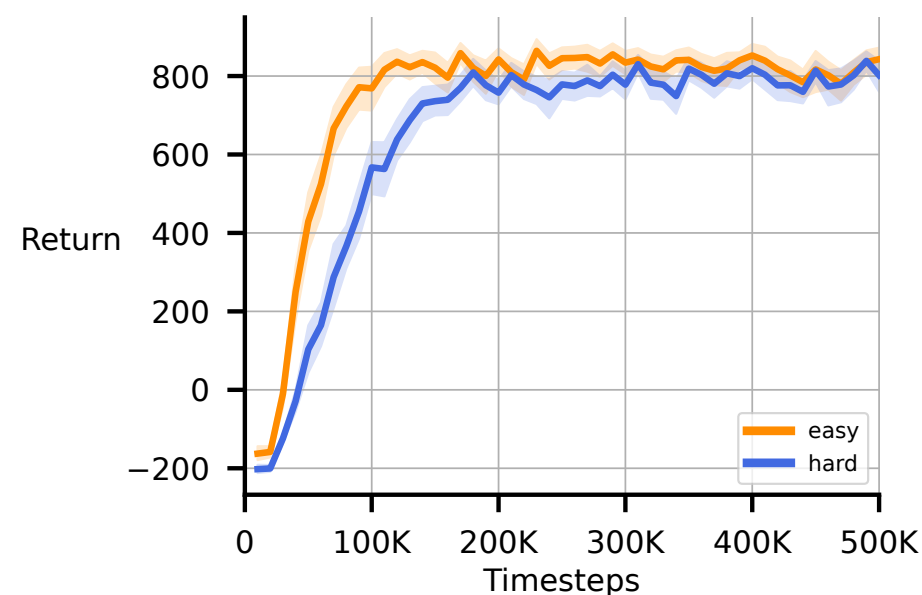
- Reacher is a simulated two-joint arm from the DeepMind Control Suite.
- The task is to reach the target as quickly as possible and remain there.
- The target appears at a random point in the plane; the hard variant uses a smaller target.

How to specify the Reacher Task?

Guiding Reward Specification

$$r_t = \begin{cases} 1 & \text{if in target,} \\ -||x_{goal} - x_{pos}|| & \text{otherwise.} \end{cases}$$

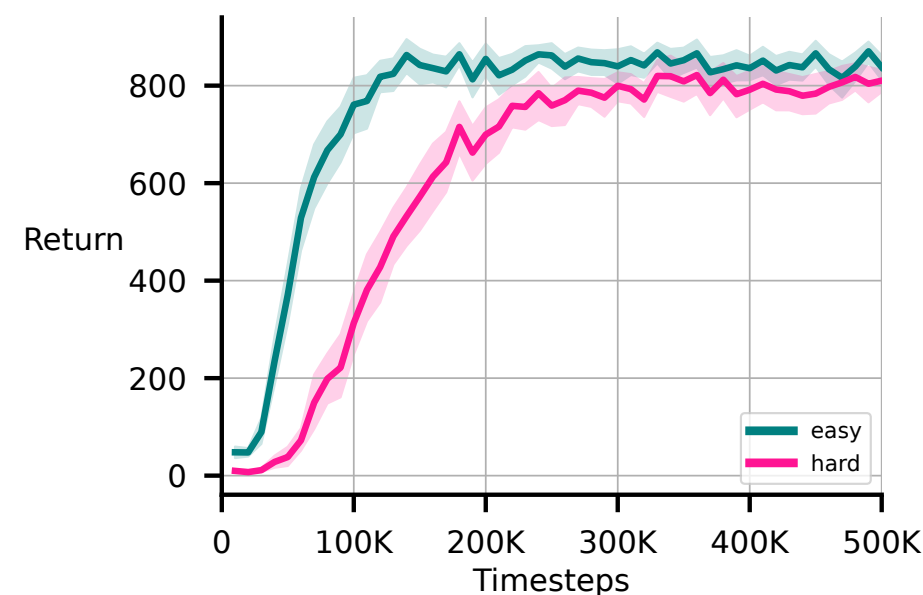
- Fixed length episodes (T=1000)



Contact Reward Specification

$$r_t = \begin{cases} 1 & \text{if in target} \\ 0 & \text{otherwise} \end{cases}$$

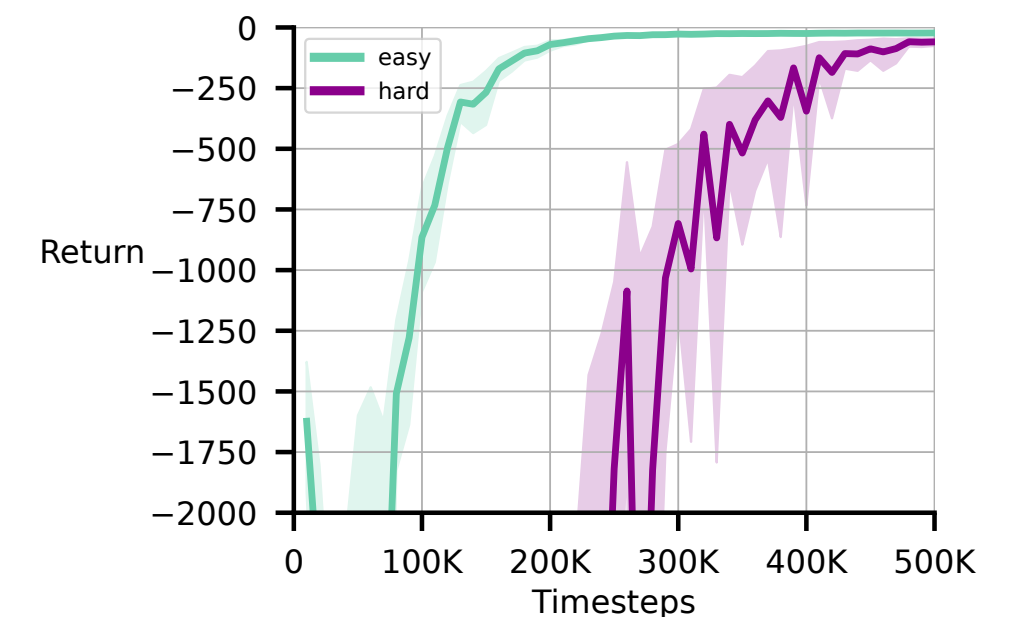
- Fixed length episodes (T=1000)



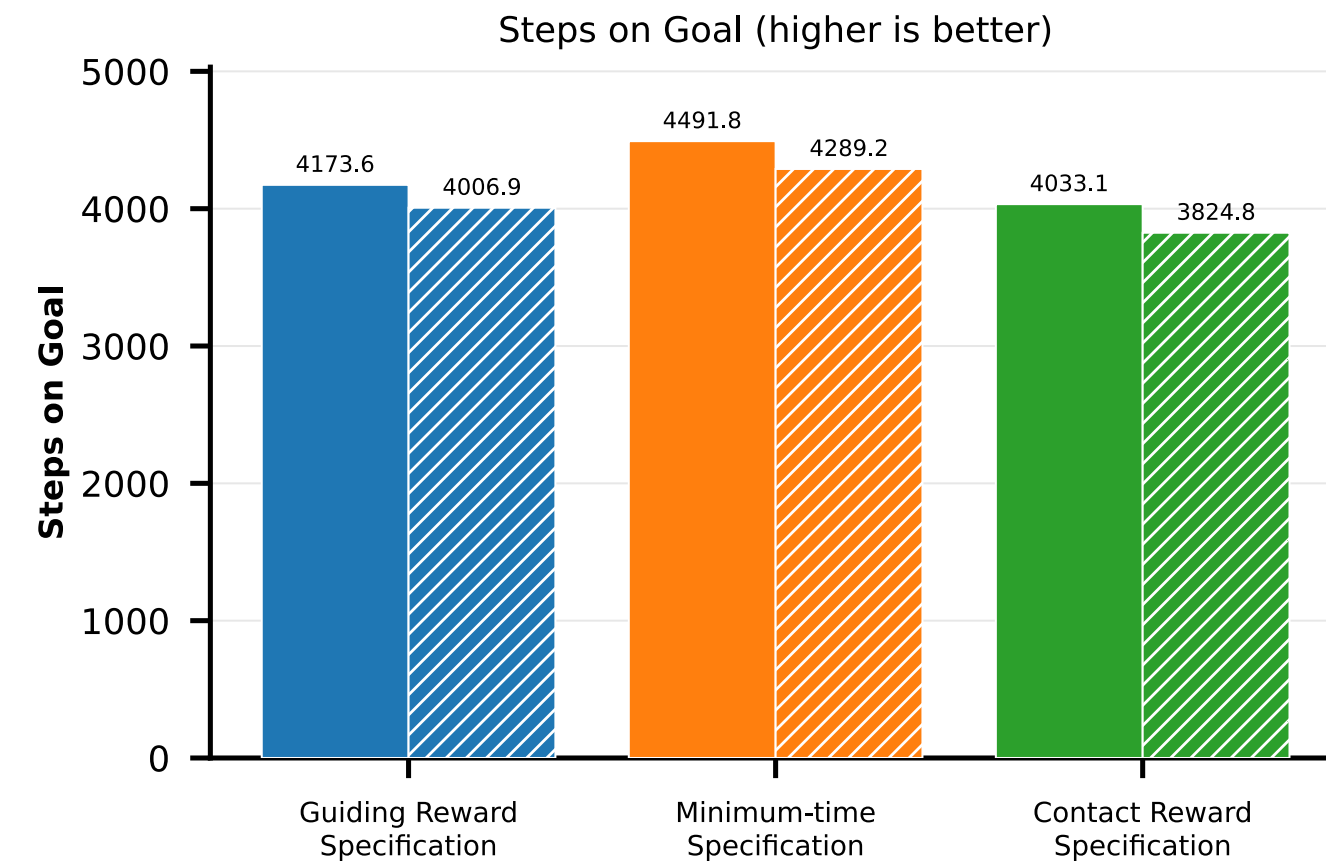
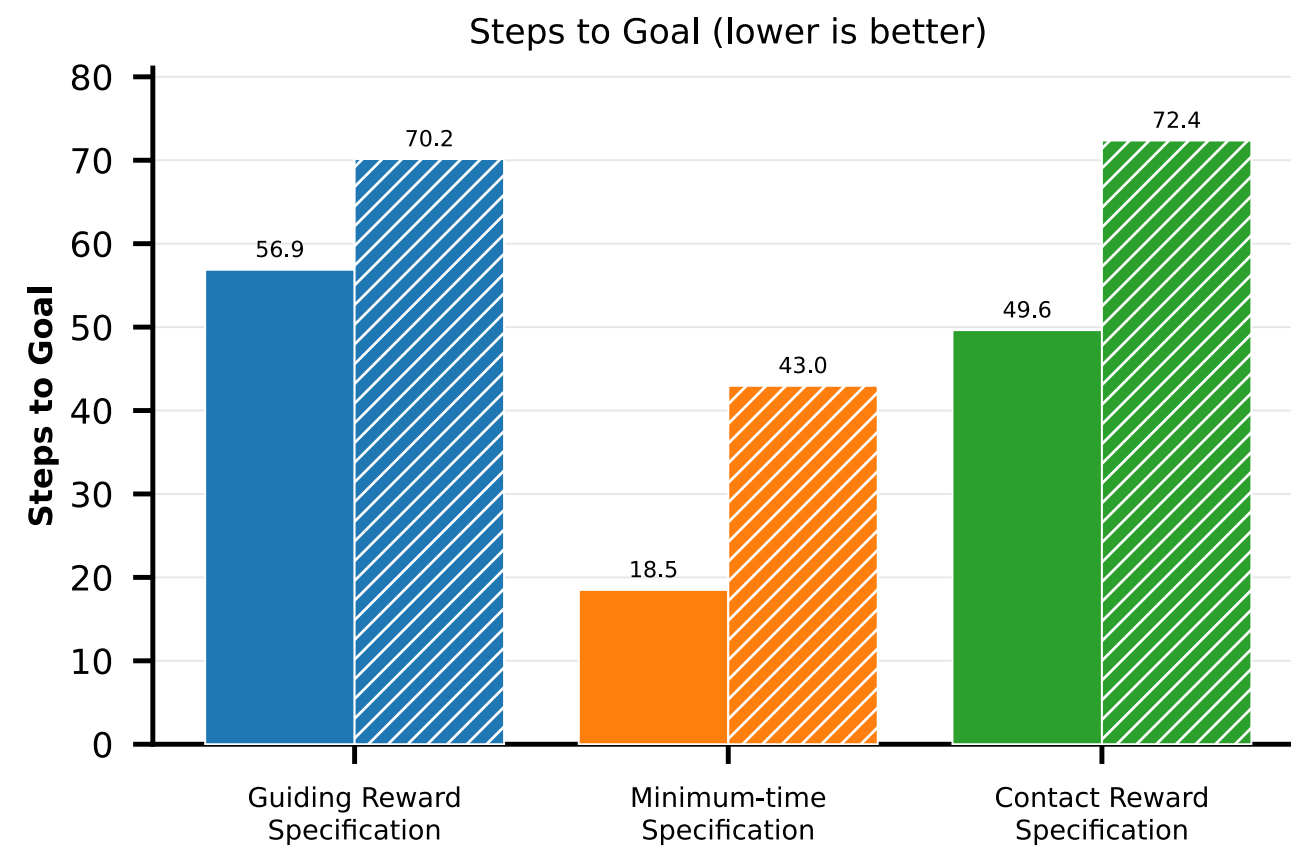
Minimum-Time Specification

$$r_t = -1 \text{ (until reaching the target).}$$

- Episode terminates upon reaching the target state with near-zero velocity
- Varying length episodes



Minimum-time policies reached the goal faster and stayed there longer.

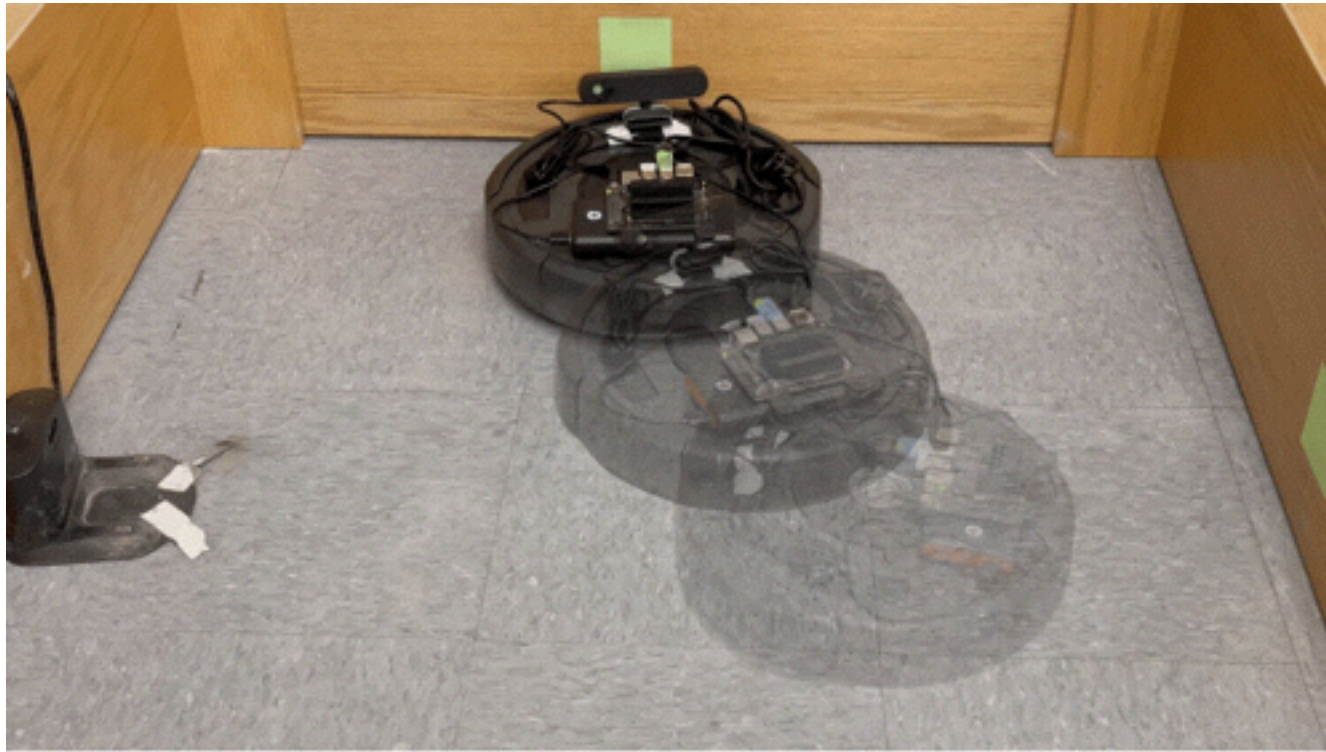


- Final policies were evaluated under one common measure, independent of the training reward: number of steps to reach the goal, and steps spent on it.

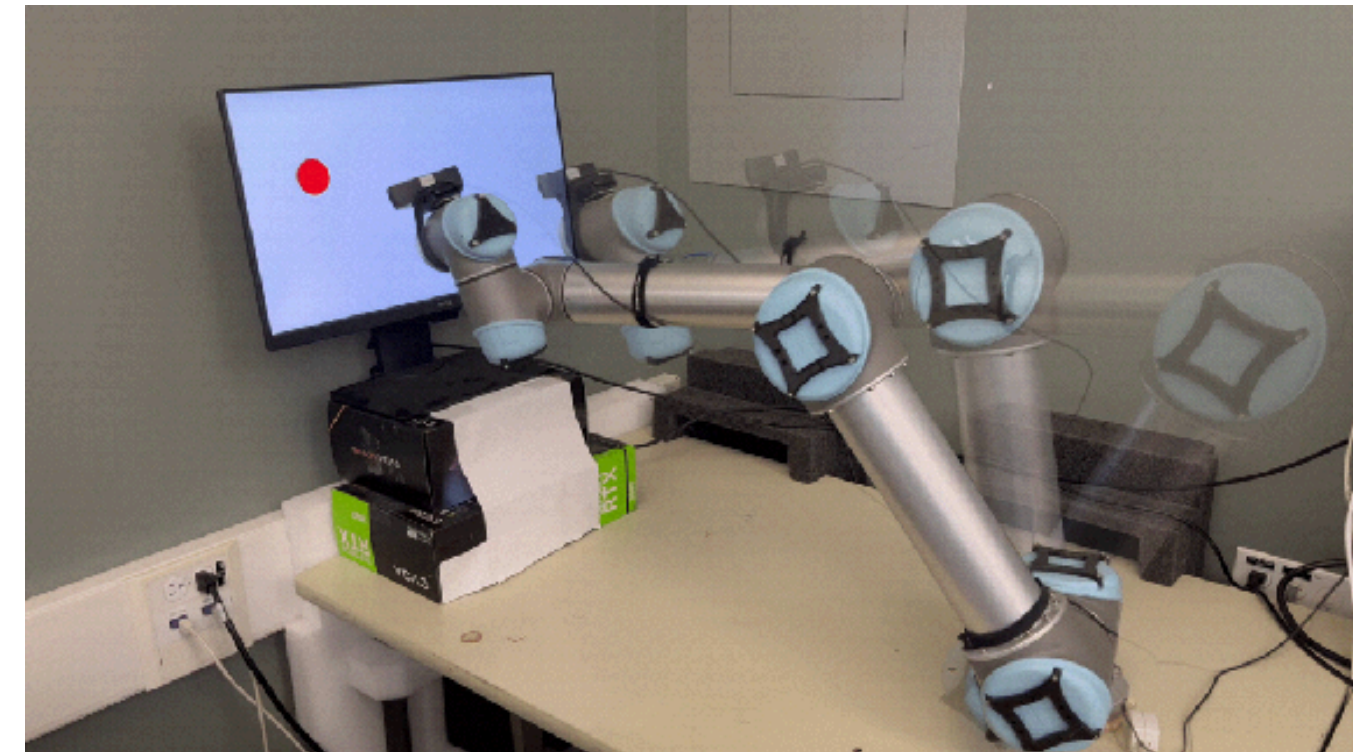
Conditions for successful learning under the minimum-time specification

- With guiding rewards, humans provide hints for exploration, which biases the learned solution.
- With the minimum-time specification, the only informative learning signal is obtained upon successfully reaching the goal.
- So learning depends on how often the initial policy reaches the goal, and the time limit is one factor that controls it: each reset places the robot at a new starting location.
- Heuristic:
For a chosen time limit, an agent using SAC can learn to solve a minimum-time task *if* the initial random policy hit the goal at least 10 times in 20K timesteps.

Four physical robots learned goal-reaching tasks from pixels under the minimum-time specification



Create-Reacher



UR5-VisualReacher



Vector-ChargerDetector



Franka-VisualReacher

Summary

- A reward of -1 per timestep and termination at the goal specifies what to do, not how.
- Final policies under the minimum-time specification matched or outperformed guiding-reward and contact-reward alternatives.
- Target hits of the initial policy can predict learning success, and the time limit is a solution parameter that controls them.
- Four physical robots learned from raw pixels within hours using the minimum-time specification.

Contribution #3: Action Value Gradient (AVG)

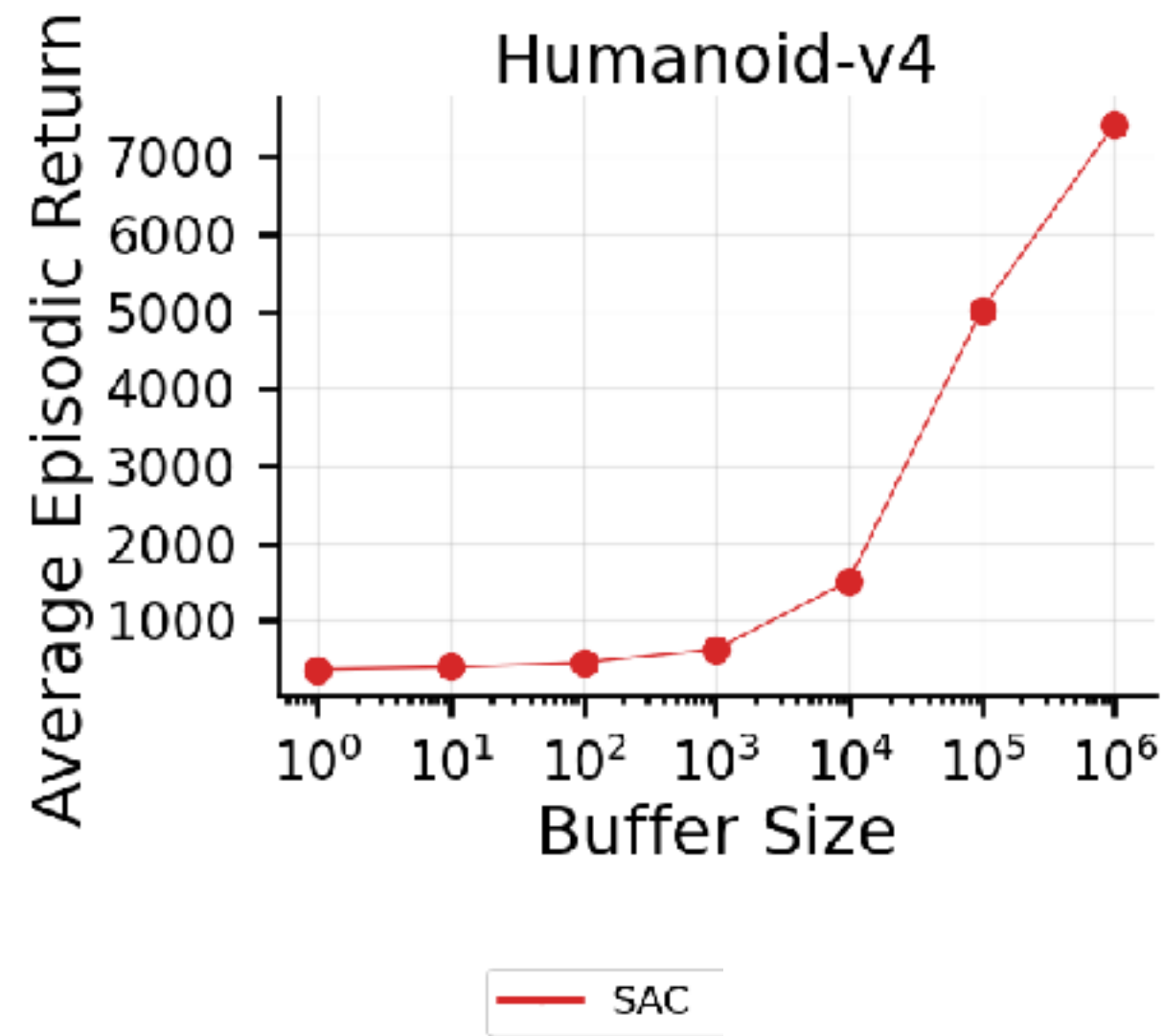
A streaming deep RL method that learns without batch updates, target networks or a replay buffer

Vasan et al. (2024b); NeurIPS

Why AVG?

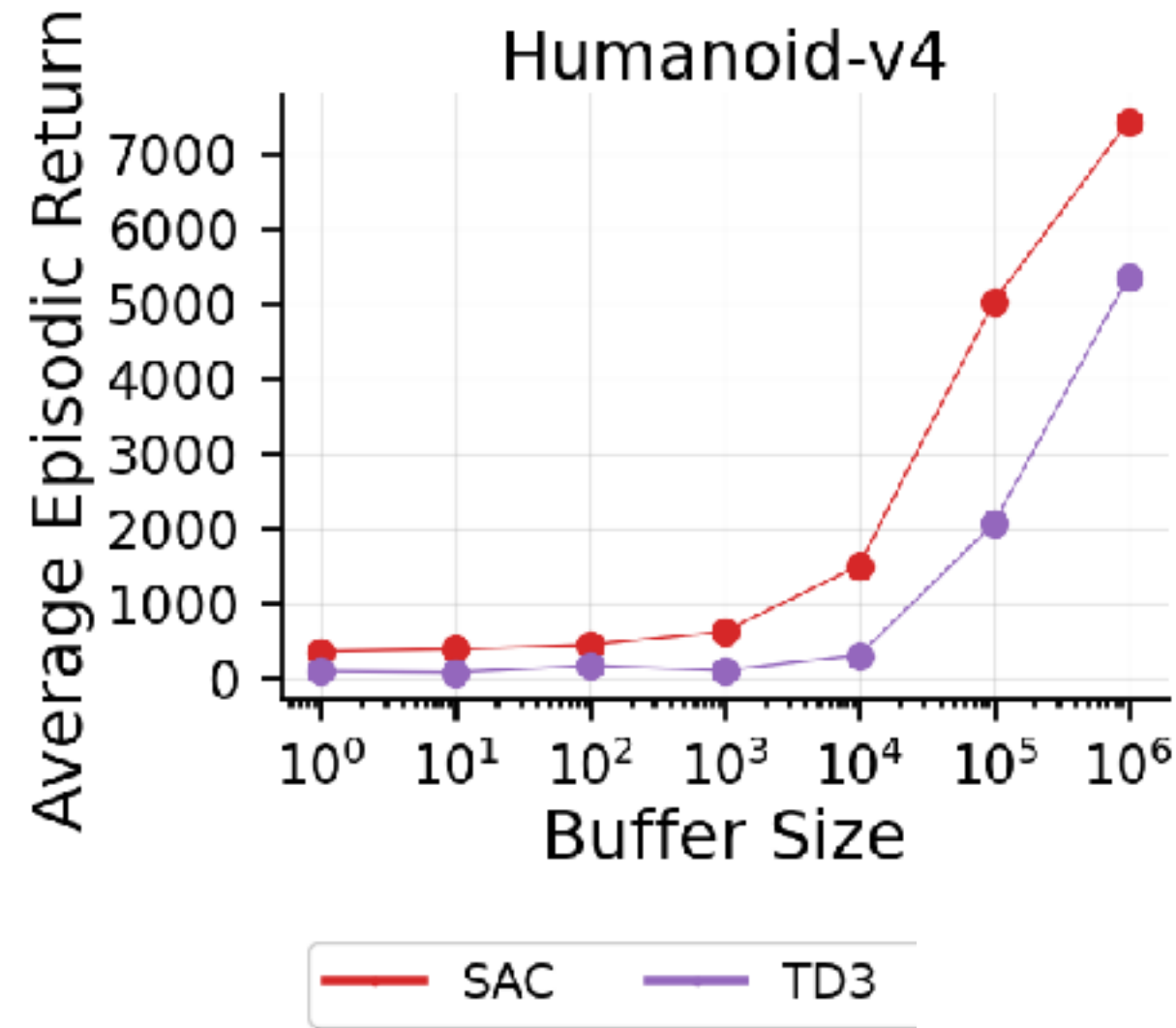
- Modern deep RL methods stabilize learning with a replay buffer, batch updates, and target networks, each a memory or computation cost an onboard computer may not afford.
- Streaming methods update from each sample as it arrives and then discard it, so their cost fits any onboard computer, but with neural networks they have been unstable.
- AVG is a streaming actor-critic method that learns with neural networks without a replay buffer, batch updates, or target networks.
- Agents using AVG learn stably with streaming updates and neural networks across multiple tasks, in simulation and on physical robots.

Impact of Replay Buffer Size on Deep RL Methods



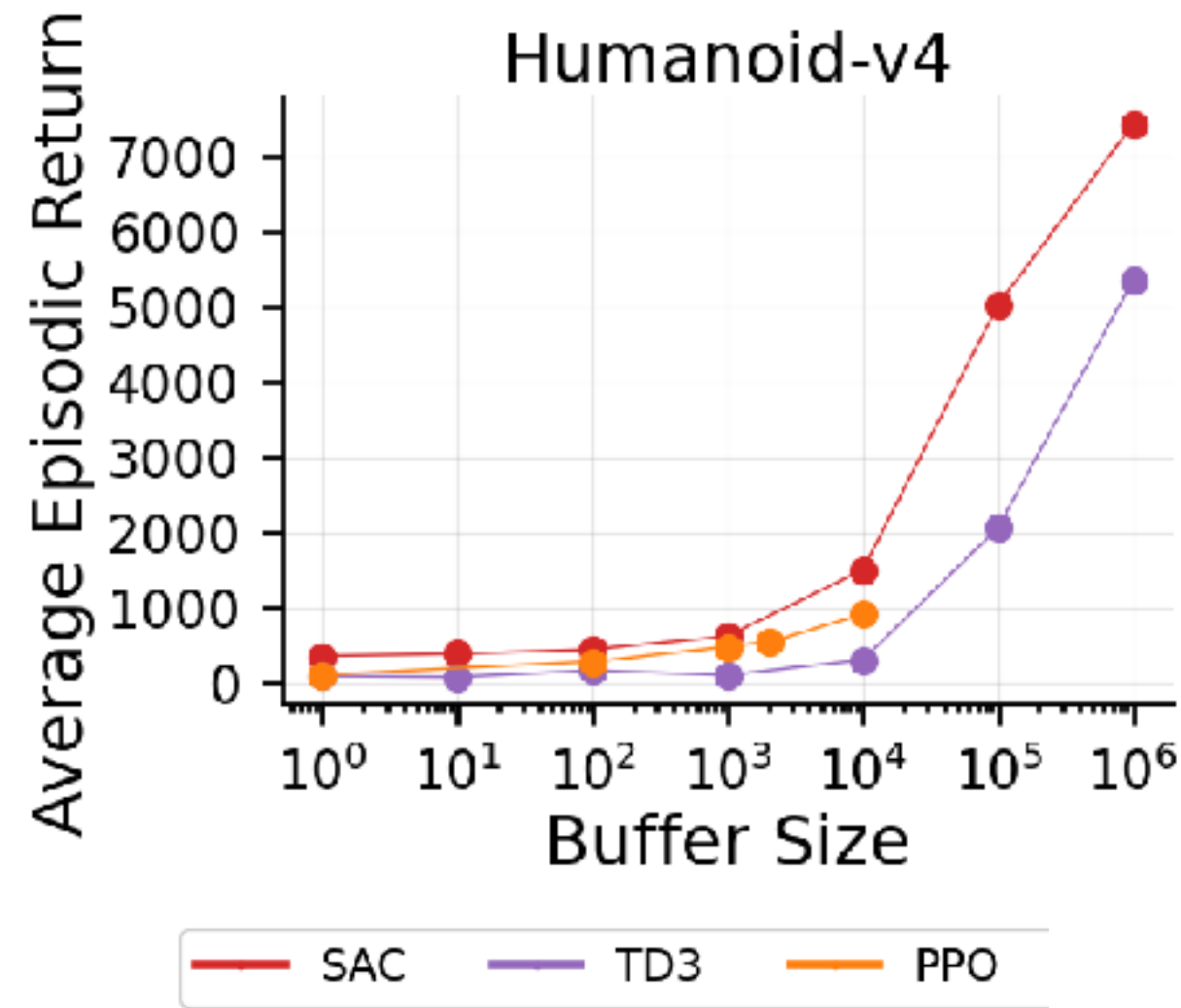
- Each agent trained for 10M timesteps on Humanoid-v4.
- Each point is the return over the last 10K timesteps, averaged over 30 runs; the x-axis is logarithmic.

Impact of Replay Buffer Size on Deep RL Methods



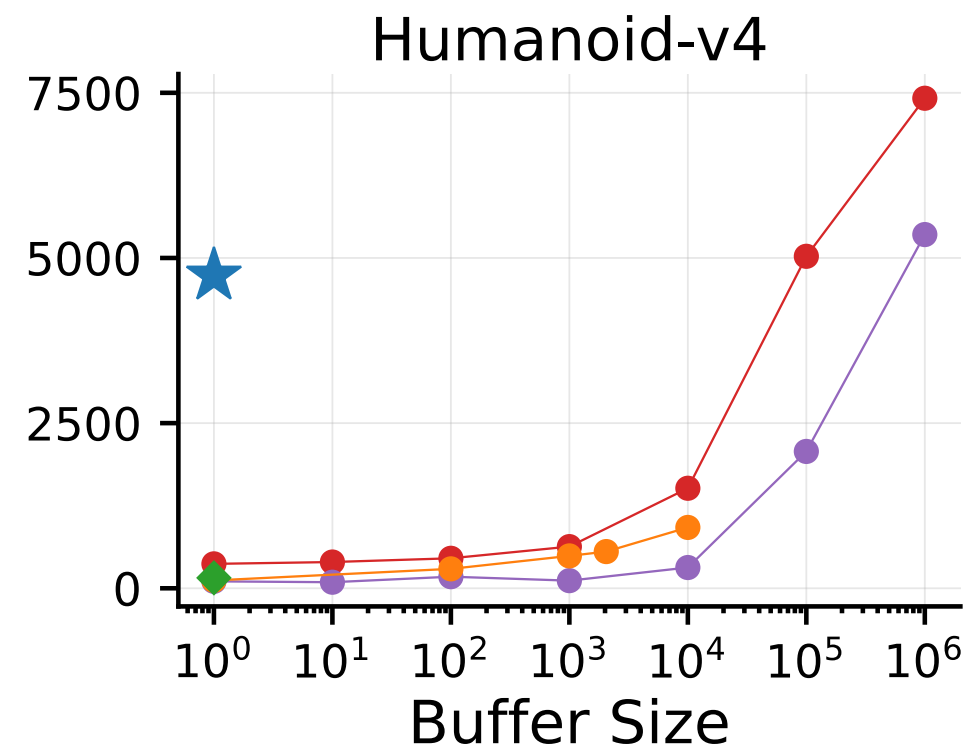
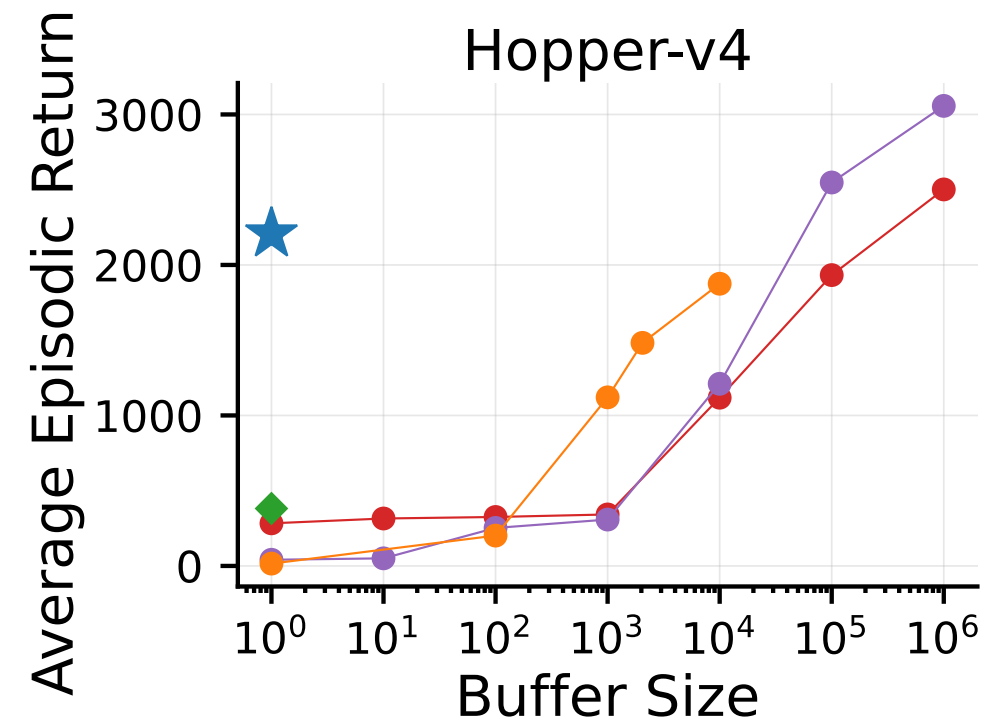
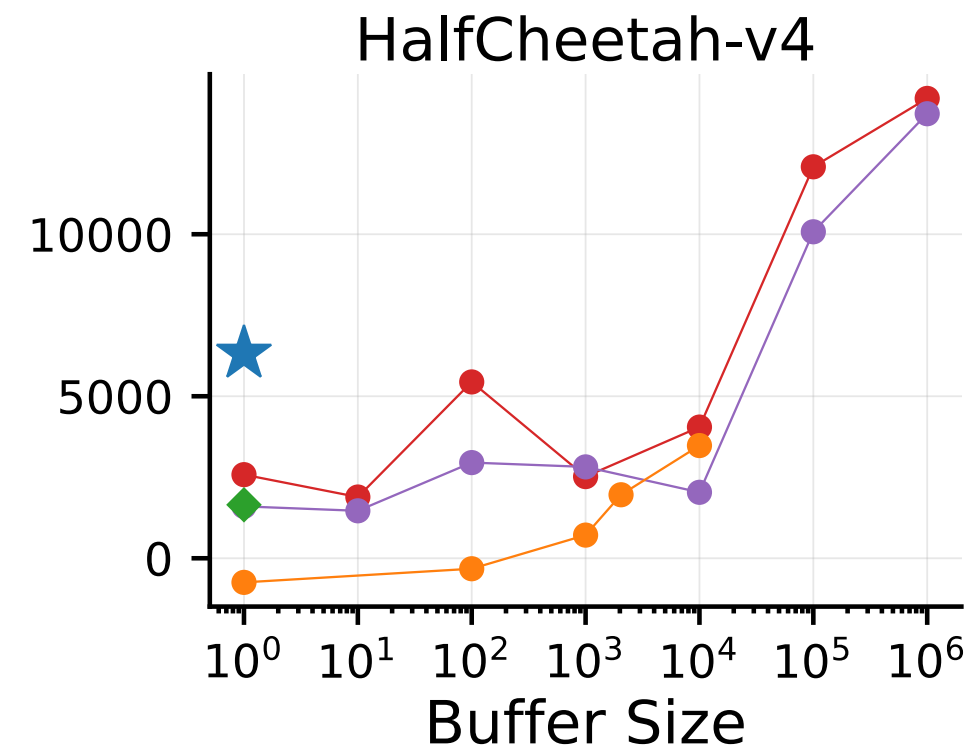
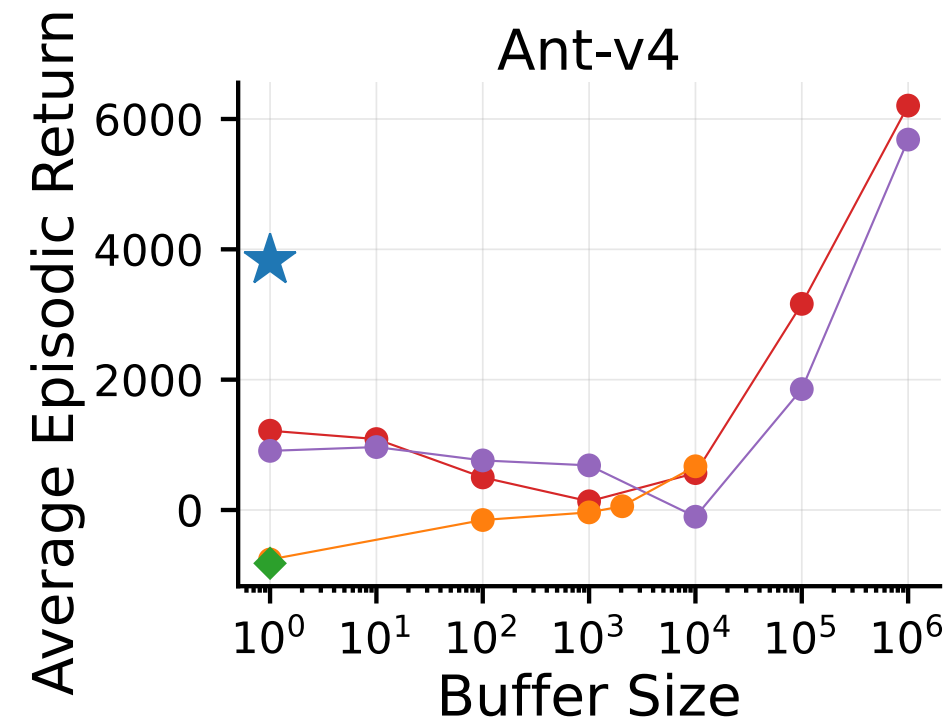
- Each agent trained for 10M timesteps on Humanoid-v4.
- Each point is the return over the last 10K timesteps, averaged over 30 runs; the x-axis is logarithmic.

Impact of Replay Buffer Size on Deep RL Methods



- Each agent trained for 10M timesteps on Humanoid-v4.
- Each point is the return over the last 10K timesteps, averaged over 30 runs; the x-axis is logarithmic.

Impact of Replay Buffer Size on Deep RL Methods



- Performance degraded as the buffer shrank from its default size.
- With buffer size set to one, agents using PPO, TD3, and SAC failed to learn.
- Agents using One-step Actor-Critic (OAC), a streaming method, also failed.
- Agents using AVG, our streaming method, learned on all four tasks.

— SAC — TD3 — PPO — OAC — AVG

Action Value Gradient (AVG)

is a streaming deep policy gradient method that does not require batch updates, target networks, or a replay buffer.

The AVG Update

Reparametrization technique is used here
to sample from a Gaussian policy $\mathcal{N}(\mu, \sigma)$

$$A_\theta = \tanh(\overbrace{\mu_\theta(S) + \sigma_\theta(S)} \epsilon) ; \epsilon \sim \mathcal{N}(0,1)$$

// Action Sampling

$$A' \sim \pi_\theta(\cdot | S')$$

Entropy regularization term

$$\delta \leftarrow R + \gamma(Q_\phi(S', A') - \overbrace{\eta \log \pi_\theta(A' | S')}) - Q_\phi(S, A_\theta)$$

// TD Error

$$\phi \leftarrow \phi + \alpha_Q \delta \nabla_\phi Q_\phi(S, A_\theta)$$

// Critic Update

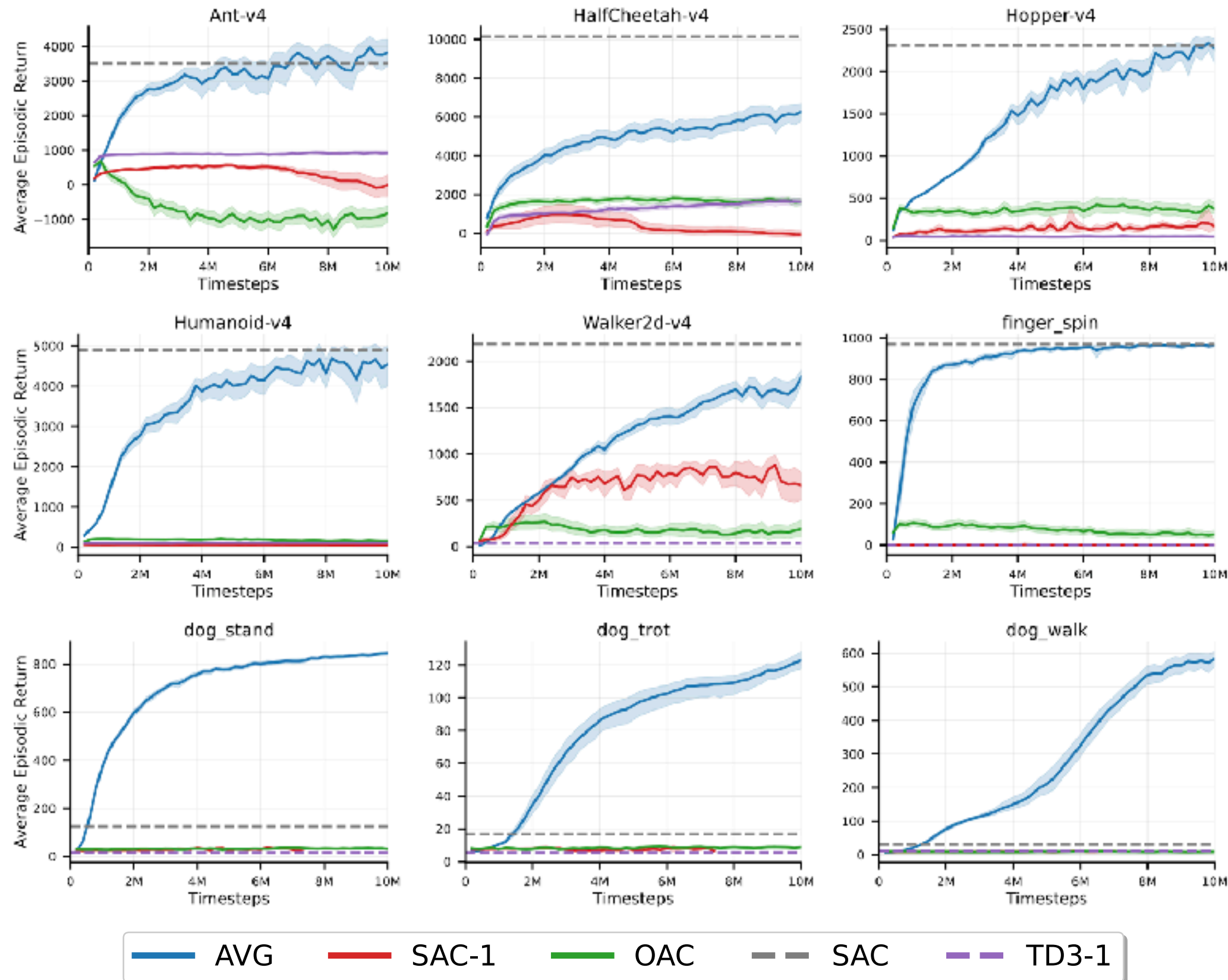
$$\theta \leftarrow \theta + \alpha_\pi \nabla_\theta (Q_\phi(S, A_\theta) - \eta \log \pi_\theta(A_\theta | S))$$

// Actor Update

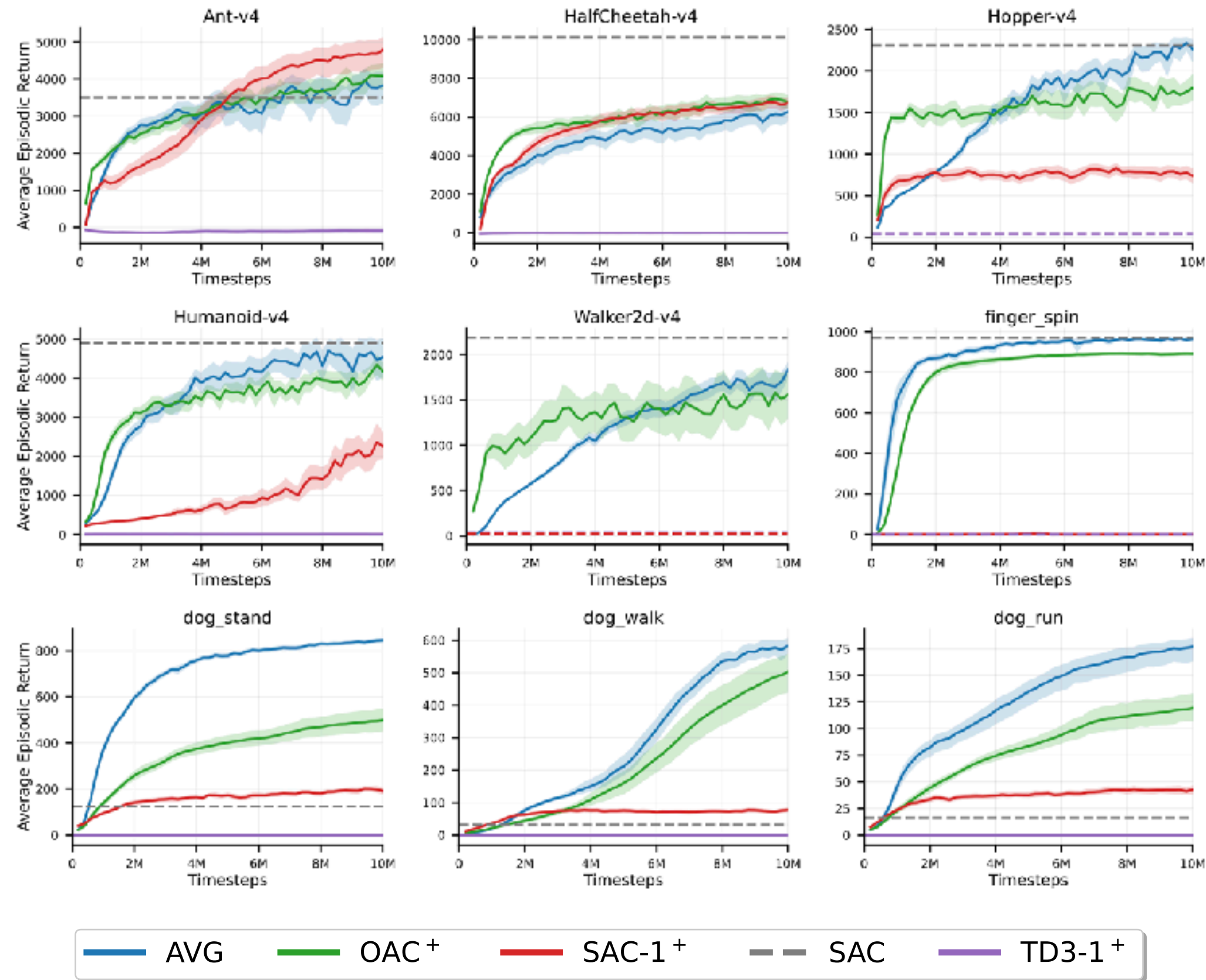
Without normalization, agents using AVG often fail to learn

- AVG uses three normalization techniques:
 - observation normalization,
 - normalizing the post-activations of the last hidden layer, and
 - temporal-difference (TD) error normalization.
- Each fixes the scale of one quantity: the inputs, the features the output layer sees, and the TD error that drives the critic update.
- Without them, agents using AVG often fail to learn.

Among the streaming methods tested, only agents using AVG learned effective policies



Normalization recovered OAC, but not SAC-1 or TD3-1



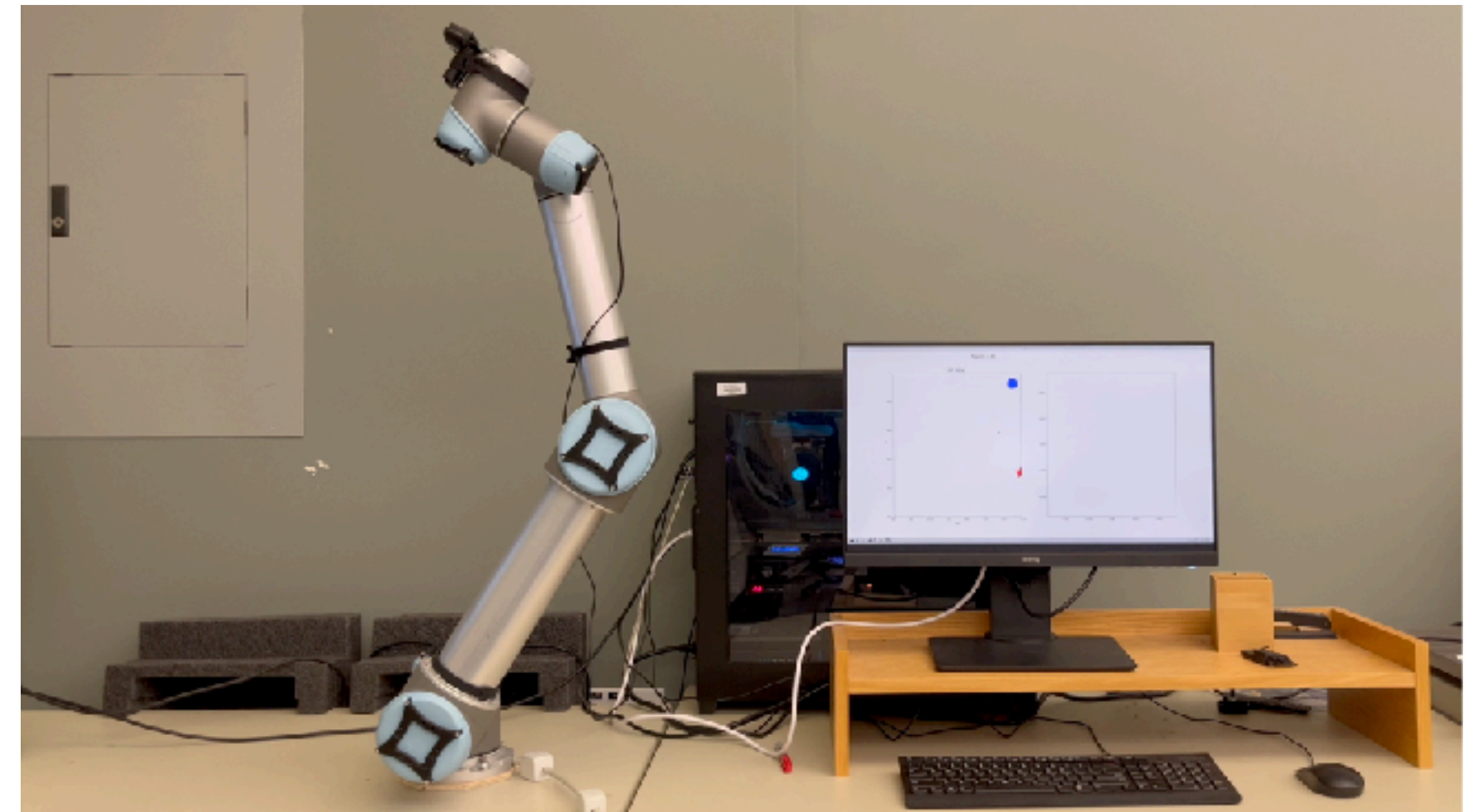
A demonstration of streaming deep RL on physical robots with AVG



Create-Mover Policy After 100K timesteps (1x)

Reward: Moving Forward

Learned Solely on Onboard Computer (Jetson Nano)



UR-Reacher-2 Policy After 400K timesteps (1x)

Reward: Distance Penalty + Precision Bonus

Summary

- AVG is a streaming actor-critic method with one update per timestep and no batch updates, target networks or a replay buffer.
- Stable learning with AVG depends on observation, penultimate-layer, and TD error normalization; without them, agents often fail to learn.
- Agents using AVG learned effective policies in simulation and on real robots, often matching the final performance of agents using batch RL methods that require vastly more compute.
- Streaming deep RL methods like AVG unlock on-device robot learning!

Conclusion

Every physical robot in this dissertation learned from scratch, in real time, from its own experience

- ReLoD supplements a robot's onboard computer with a remote one when a network link is available, at the cost of a more complicated setup.
- AVG brings real-time deep RL within the robot's onboard computer, and opens streaming learning to further advances, such as model-based RL.
- For the goal-reaching tasks studied, a reward of -1 until the goal was reached, detected from onboard sensing, produced better final policies than a guiding reward.
 - Learning with sparse rewards still needs better exploration methods.
- Every physical robot in this dissertation learned from scratch within hours of experience.

Toward continual adaptation and long-term autonomy

- Three decades of demonstrations, and real-time RL never became the norm.
- *My contributions lower the barriers to entry:* simple rewards a robot computes itself, learning within a wall-clock budget, and learning within the robot's own computational resources.
- A robot that learns from its own runtime experience can improve its behavior over its operational lifetime and adapt to conditions that were never anticipated.
- What remains is continual adaptation: addressing nonstationarity, retention of what was learned, and choosing what to learn next; and these questions can be studied now with robots as a platform.

Thank you!



Rupam Mahmood



Yan Wang



Fahim Shahriar



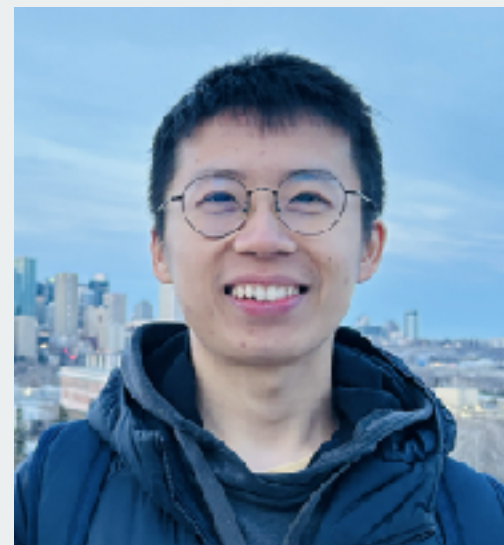
James Bergstra



Martha White



Mohamed Elsayed



Jiamin He



Alireza Azimi



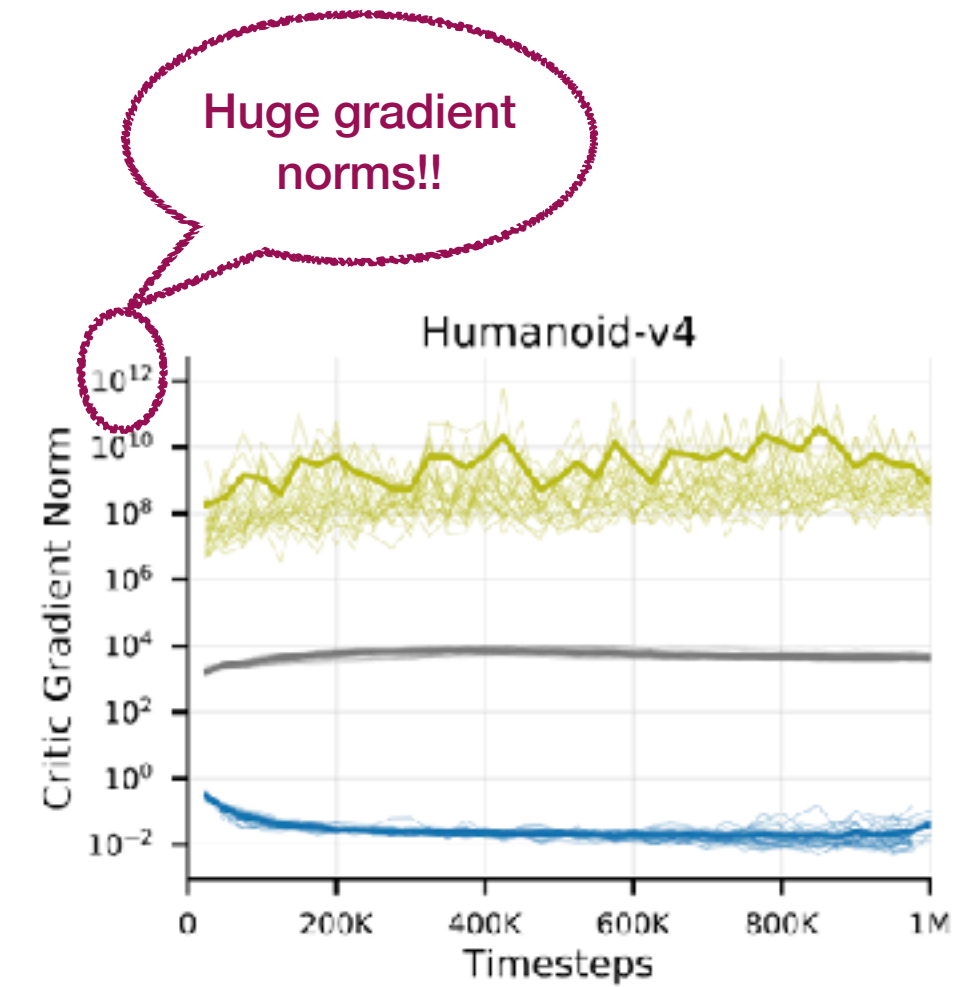
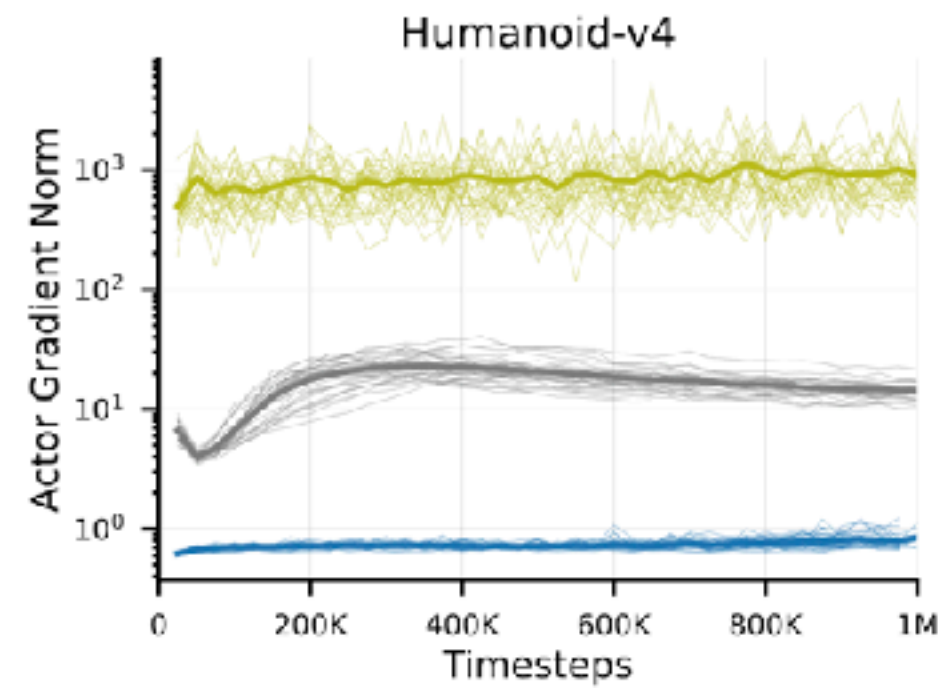
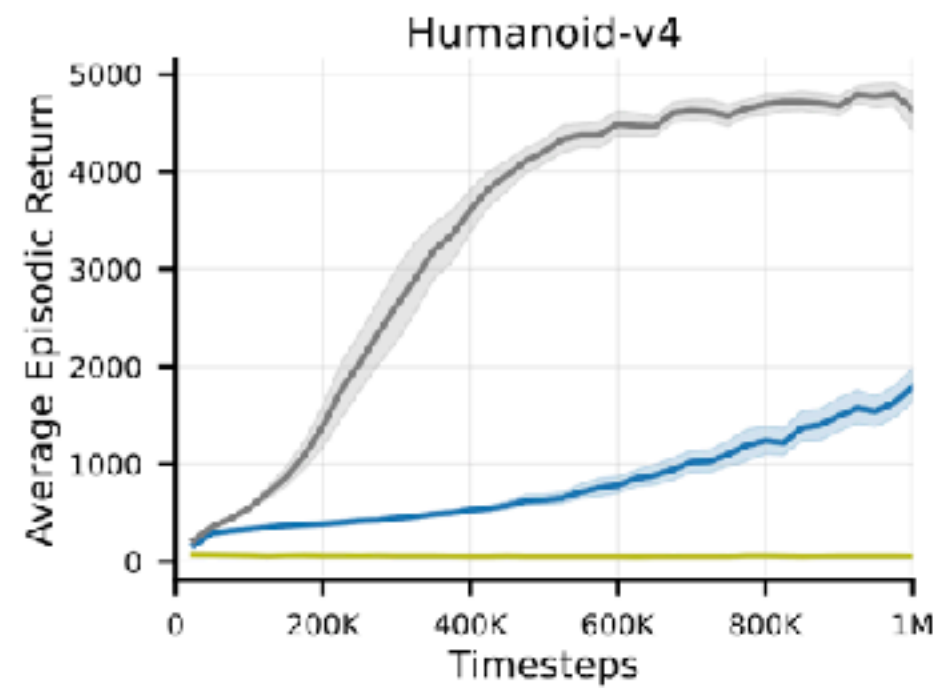
Martin Jagersand



Colin Bellinger



Instability Without Normalization and Scaling



— AVG — AVG⁻ — SAC

Ablation Study of Normalization & Scaling Techniques

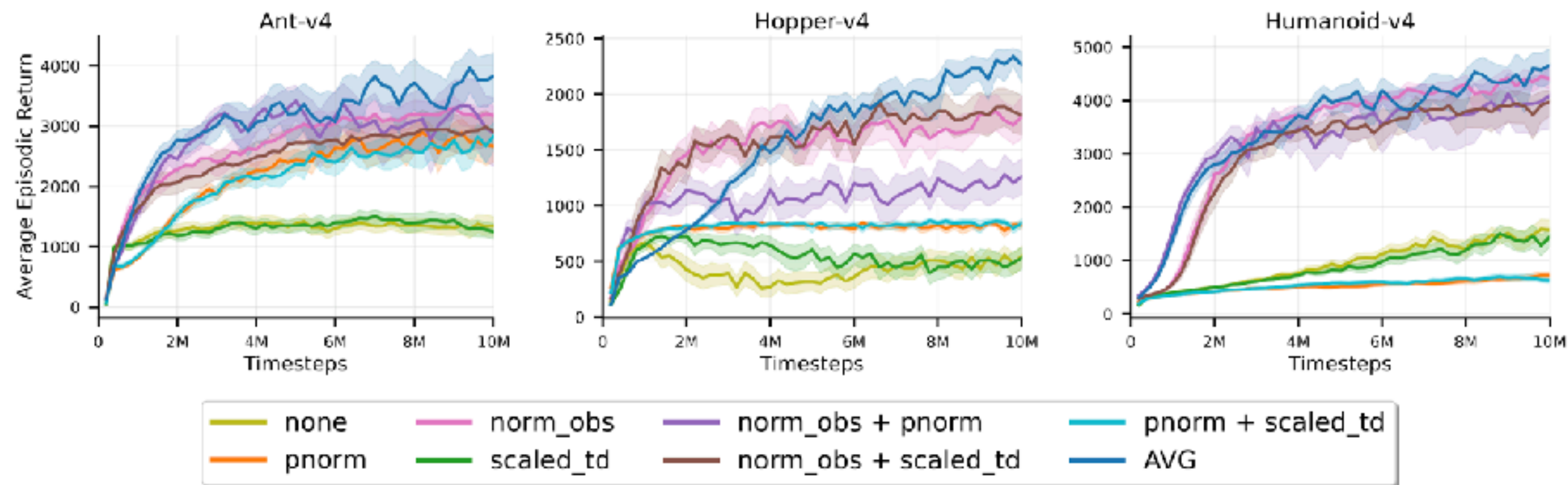


Figure 6: Ablation study of normalization and scaling techniques used with AVG. We plot the learning curves of the best hyperparameter configurations for each task variant. Each solid learning curve is an average of 30 independent runs. The shaded regions represent a 95% confidence interval.

Does AVG benefit from target networks?

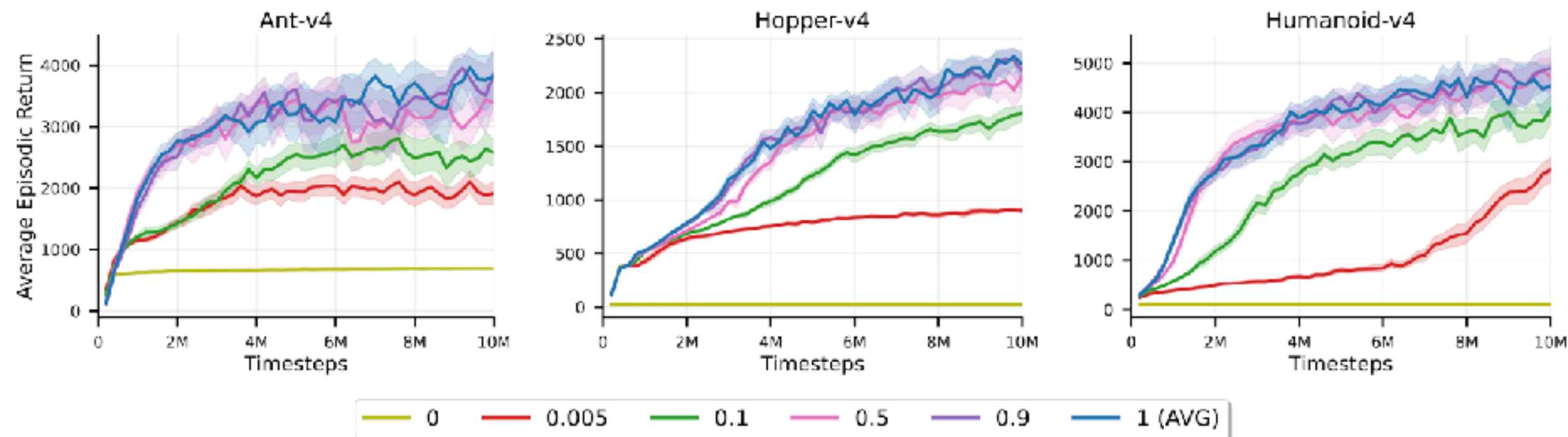


Figure 8: Impact of target Q network on AVG for different values of τ , which represents the Polyak averaging coefficient. Here, $\tau = 0$ corresponds to a fixed target network, and $\tau = 1$ indicates that the current Q-network and the target network are identical, that is, not using a target network.

Robot Experiments

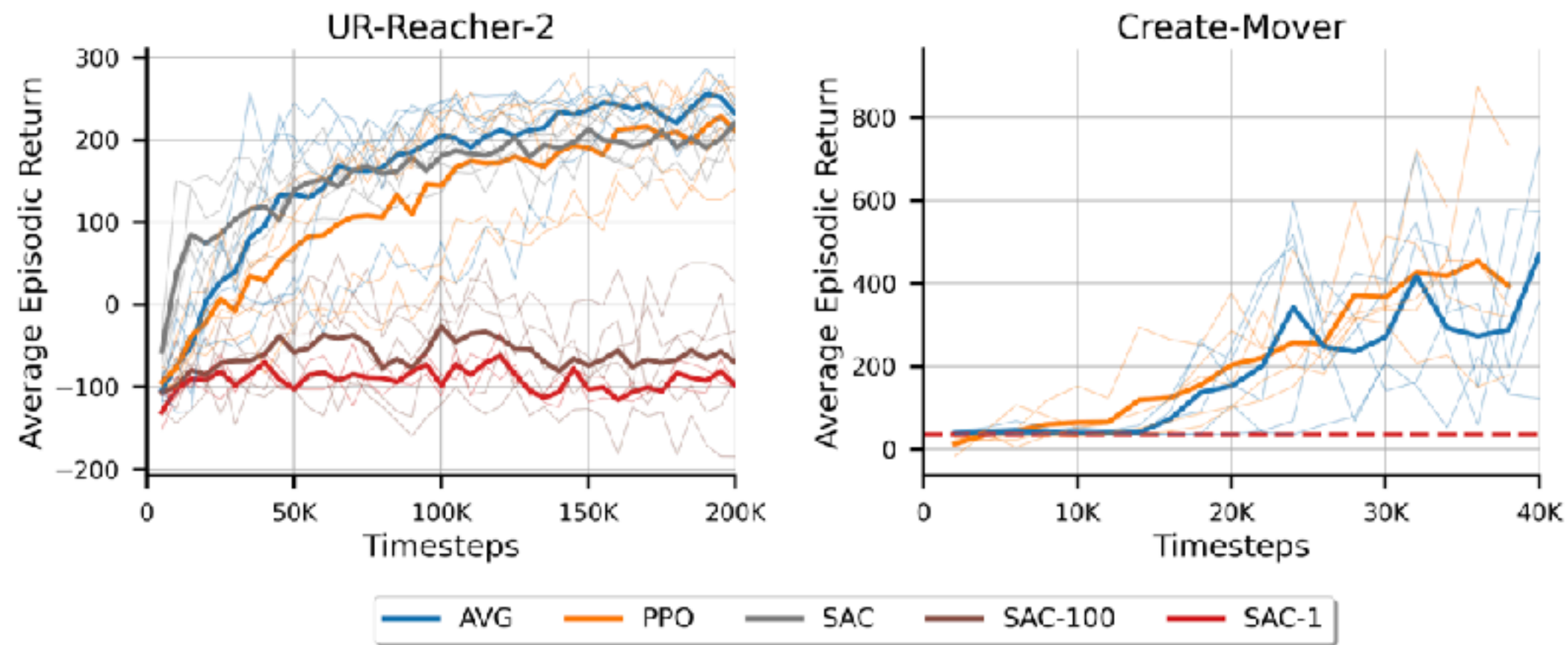
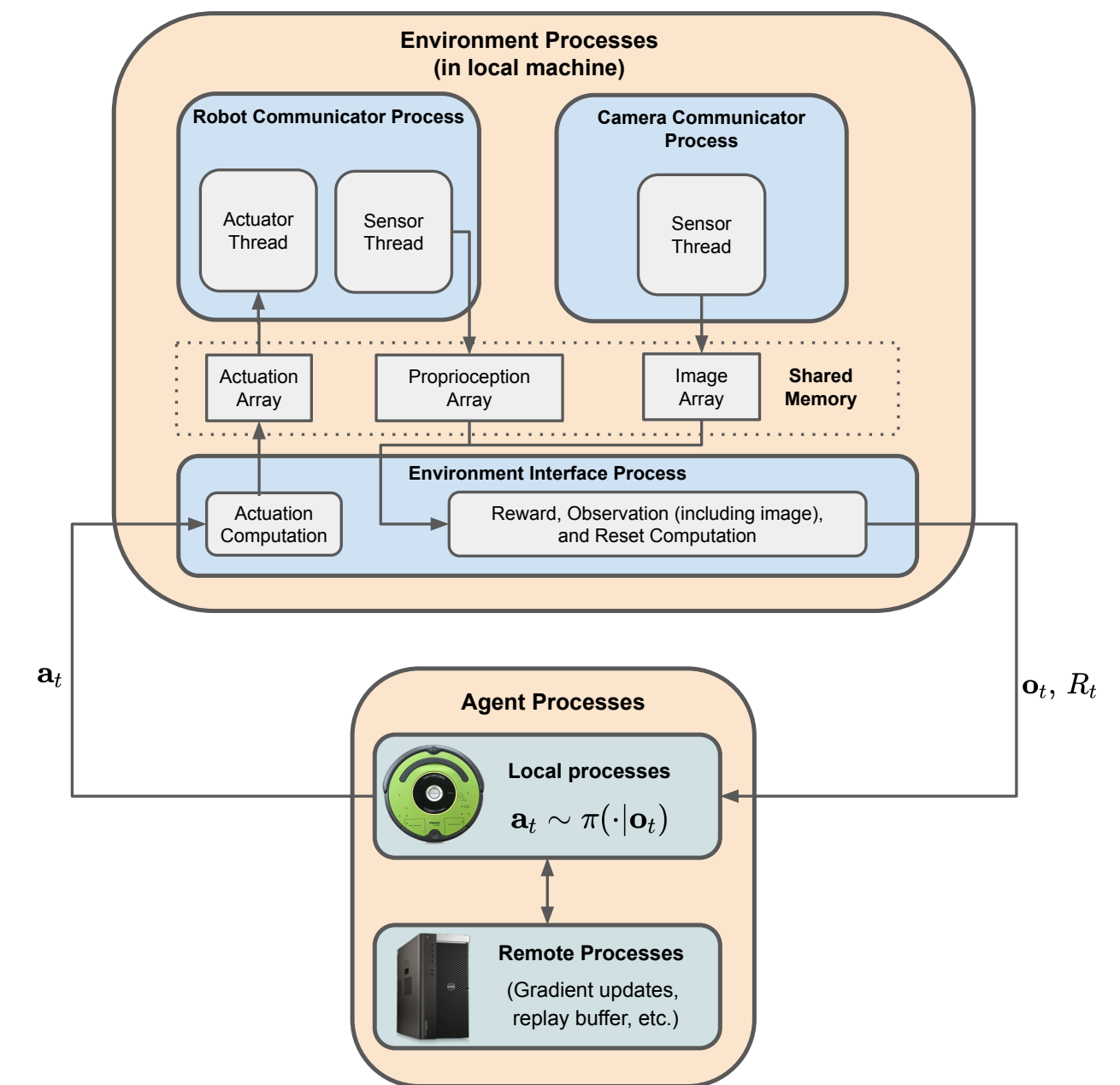


Figure 21: Learning curves on Robots. Comparison of AVG with full PPO & SAC. Note that running SAC and SAC-100 onboard for the Create-Mover task is computationally infeasible.

ReLoD: The Remote Local Distributed Framework

- ReLoD distributes an agent's computation between the robot's local computer and a remote workstation.
- The framework implements SAC, an off-policy method with expensive updates, and PPO, an on-policy method with cheaper ones.
- Three modes place the computation: Remote-Only, Remote-Local, and Local-Only.



Takeaways from our study on distributing the computations of an RL method

- Remote-Local learning matched learning on the workstation alone, given a high-bandwidth network link.
- Computing actions remotely and sending to the robot over WiFi impacts learning performance;
 - Latency in action execution is detrimental to learning.
- SAC, which stores every transition, made better use of limited robot experience than PPO, which discards each rollout.